

Review

Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models

Md. Shazzad Hossain Shaon  and Mst Shapna Akter *

Department of Computer Science and Engineering, Oakland University, Rochester, MI 48309, USA; shaon@oakland.edu

* Correspondence: akter@oakland.edu

Abstract

Software vulnerabilities pose significant risks to the security and reliability of modern systems, making automated vulnerability detection an essential research area. Traditional static and rule-based approaches are limited in scalability and adaptability, motivating the adoption of data-driven methods. In this survey, we present a comprehensive review of Machine Learning (ML), Deep Learning (DL), and Large Language Models (LLMs) techniques for vulnerability detection. We analyze recent advances in feature representation, fine-tuning strategies, generative approaches, and prompt engineering, while highlighting their ability to capture both syntactic and semantic properties of source code. Furthermore, we examine commonly used evaluation metrics and provide a critical discussion of key challenges, including the lack of large-scale real-world datasets, limited vulnerability coverage, class imbalance, interpretability gaps, hallucination, and high computational costs. To address these issues, we outline promising future research directions, such as neuro-symbolic hybrid methods, parameter-efficient fine-tuning, continual learning, cross-language generalization, and explainable AI for vulnerability detection. Unlike previous studies, the present work explores learning paradigms from ML to LLMs using comprehensive evaluation criteria that highlight analytical capability, feature interpretability, and code-context comprehension. By combining these factors, our study addresses the methodological gap between classic feature-based approaches and current LLM-driven reasoning frameworks, providing beneficial insights to develop robust, scalable, and trustworthy software vulnerability detection systems.

Keywords: software vulnerabilities; large language model; system security; feature representation; deep learning; software



Academic Editor: George Angelos Papadopoulos

Received: 13 October 2025

Revised: 2 November 2025

Accepted: 7 November 2025

Published: 14 November 2025

Citation: Shaon, M.S.H.; Akter, M.S. Modern Approaches to Software Vulnerability Detection: A Survey of Machine Learning, Deep Learning, and Large Language Models.

Electronics **2025**, *14*, 4449. <https://doi.org/10.3390/electronics14224449>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Modern computer systems serve as the core for current technological infrastructure, ensuring both customers and businesses interact and operate seamlessly. However, exploitable software vulnerabilities remain a continuous danger to system security and user safety. High-profile incidents such as the “Heartbleed” [1] and “Shellshock” [2] vulnerabilities, along with the 2017 Apache Struts breach that exposed confidential financial data of 143 million users demonstrate the widespread effect and significant nature of those vulnerabilities [3–6]. Despite the development of several detection techniques, these cases happened mostly due to previous approaches’ failure to discover advanced vulnerabilities early of their exploitation. This underlines a crucial research gap: the need for more

adaptable and intelligent systems that can reason about semantic and contextual connections inside source code, rather than depending simply on rule-based or pattern-matching tactics. A preventative and effective technique for preventing attacks includes identifying software vulnerabilities during deployment. As shown in Figure 1, software vulnerability analysis techniques can be broadly classified into three main categories: static, dynamic, and hybrid approaches, where static vulnerability detection approaches evaluate the program without executing it, depending heavily on the source code. Common approaches include rule- or template-based analysis [7–9], code similarity identification [10,11], and symbolic execution [12–14]. These methods are capable of covering large code sections and detecting possible faults early in the development process. However, they have been reproved for developing a high proportion of false positives, which can lead to extra human intervention in validating observations [15]. Dynamic analysis approaches, for example fuzz testing [16,17], and taint analysis [18,19], evaluation software during execution to identify flaws that static methods might overlook. Although dynamic techniques are more effective at detecting runtime behaviors and vulnerabilities that are dependent on particular execution environments, they sometimes have limited code coverage and can fail to identify hidden defects in seldom performed routes. To address the limitations inherent in both static and dynamic methods, hybrid approaches have been developed. These combine static code analysis with runtime monitoring and testing to leverage the strengths of both paradigms. The integration aims to improve detection accuracy, reduce false positives, and enhance code coverage, thereby offering a more comprehensive solution for software vulnerability analysis.

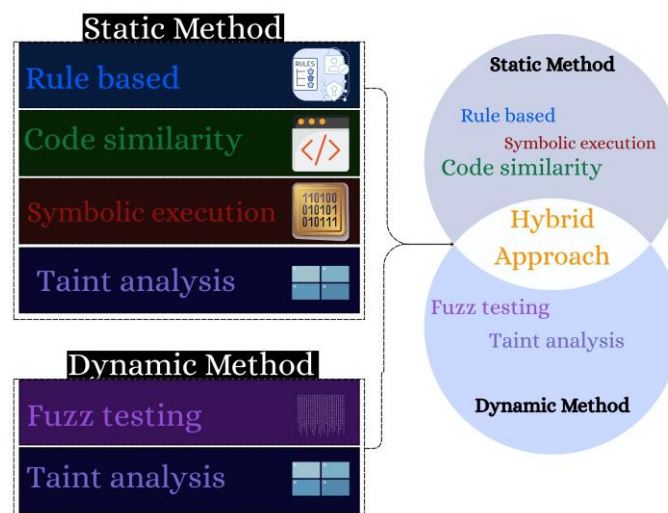


Figure 1. An overview of software vulnerability detection techniques. Static techniques analyze code before execution, whereas dynamic approaches identify flaws during runtime. Hybrid techniques utilize components from both paradigms to capitalize on their respective strengths in reliability, visibility, and identifying effectiveness.

Recent advancements in data-driven vulnerability discovery have introduced novel approaches that go beyond traditional static and dynamic analysis techniques. Over the past decade, vulnerability detection research has advanced from traditional Machine Learning (ML) models, which rely on manual feature extraction, to Deep Learning (DL) architectures capable of automatically learning hierarchical code representation. As the size and complexity of software increased, DL-based approaches became more successful at capturing contextual dependencies inside source code. Building on this achievement, the introduction of Large Language Models (LLMs) is a logical next step, since these models expand DL capabilities by including large-scale pretraining, code semantics comprehen-

sion, and reasoning skills. By leveraging pattern recognition and Artificial Intelligence (AI) methods [20–37], researchers have been able to automate vulnerability detection processes, improving both scalability and efficiency. Unlike previous strategies, which depended heavily on handcrafted rules and shallow heuristics, these innovative methods allow systems to learn directly from vast datasets of susceptible and non-vulnerable code. As a result, ML, DL, and LLM-based frameworks provide formidable options for automatic, accurate, and economically viable software vulnerability detection. Furthermore, the development of neuro-symbolic methods and contrastive learning strategies shows an evolution toward hybrid systems that integrate statistical learning with logical reasoning, lowering false positives and improving interpretability. These advancements represent a fundamental shift in vulnerability detection research, with an emphasis not just on discovering defects but also on understanding vulnerability patterns, root causes, and mitigation measures. As a result, the use of ML, DL, and LLM-based methodologies represents a significant move toward more robust, proactive, and intelligent security assessment solutions for current software environments. Therefore, this study provides a comprehensive review of recent models and approaches for software vulnerability detection, with a particular focus on the patterns in source code that these models exploit. In addition to analyzing conventional machine learning-based techniques, the review also covers deep learning architectures and large language model-driven frameworks that are increasingly being adopted to capture both syntactic and semantic vulnerability indicators.

2. Related Reviews

This section discusses currently available research on software vulnerability detection. Several strategies have been proposed and intensively researched in the field of software engineering for discovering shortcomings and vulnerabilities in source code. Early research concentrated mostly on static and dynamic analysis techniques that employed manually generated rules, symbolic execution, or testing frameworks to detect security issues. Over time, the research community has shifted toward more data-driven methods, incorporating pattern recognition, machine learning, and deep learning models to automate vulnerability discovery. Shahriar et al. [24] explored several code analysis techniques, classifying current methodologies according to their methodological focus and detection scope. Malhotra et al. [25] performed a study of software defect prediction approaches that used ML techniques, highlighting their benefits and limitations in detecting fault-prone code sections. In a more comprehensive assessment, Liu et al. [26] provided a broad overview of vulnerability detection research, focusing on works that used both traditional code analysis and ML-based algorithms. Ghaffarian et al. [21] conducted an in-depth review of vulnerability detection utilizing classical machine learning approaches, highlighting features based on software metrics, expert-defined patterns, and anomaly detection. Following that, another review [27] provided a succinct overview of related findings. From a different perspective, [28] performed a thorough review that identified parallels between programming languages and natural languages and explained how such comparisons influence model design. More recently, several surveys have concentrated on deep learning-based vulnerability detection. Uddin et al. [29] introduced the “Vulnerability Detection Lifecycle” framework to systematically analyze deep learning models across stages like data construction, code representation, and deployment. Hanif et al. [30] executed a comprehensive literature evaluation of 90 materials published between 2011 and 2020, providing two different taxonomies for classifying vulnerability detection studies. Harzevili et al. recently published a thorough systematic literature analysis, providing wide insights on the use of machine learning for software vulnerability detection across multiple data types including source code, binaries, and commit metadata—over a long period of time (2011–2024) [31].

Zeng et al. [32] executed their survey based on four recent “game changer” endeavors. While their methodology recognizes crucial research areas coming from these “game changers”. Lomio et al. [33] evaluated the effectiveness of an existing ML-based SVD method for commit-level detection. However, their selection of just 9 domains seems ambiguous. Liu et al. [34] conducted a survey that compared and analyzed strategies for identifying XSS vulnerabilities. Chakraborty et al. [35] carried out an empirical investigation to assess the robustness of DL-based vulnerability detection approaches when applied to real-world datasets. Their evaluation revealed a critical limitation: although existing DL-based methods often demonstrate strong performance in controlled or benchmarked environments, their effectiveness tends to deteriorate significantly in real-world scenarios. This performance gap highlights the challenges of generalizability, where models trained on curated or synthetic datasets fail to capture the complexity, diversity, and noise inherent in actual software projects. The study underscores the importance of evaluating vulnerability detection methods not only on benchmark datasets but also across diverse, real-world software repositories, thereby ensuring that proposed techniques remain reliable and scalable in practice. Zheng et al. [36] conducted an experiment to determine the impact of various ML methodologies on vulnerability detection performance. Their findings demonstrate that the attention mechanism improves vulnerability identification by allowing models to capture fine-grained contextual relationships in source code. In contrast, the use of transfer learning did not result in significant gains in model performance. This result implies that while transfer learning has been effective in other domains such as natural language processing (NLP) and computer vision, its benefits are less evident in software vulnerability detection, possibly due to the domain-specific characteristics of programming languages and security flaws. In recent times, Zhao et al. [37] proposed Yama, utilizes PHP opcode principles to perform context and path-sensitive intraprocedural analysis. It incorporates accurate type inference, dynamic language structures, and concrete execution for executing challenging built-in functions. This study demonstrates the importance of opcode-level semantic reasoning in enhancing static vulnerability detection precision in dynamic web languages. In another study. In another, Ji et al. [38] introduced Artemis, integrates GPT-4-based source/sink identification, implicit call graph construction, rule-driven taint propagation, and path-condition-based false positive pruning. However, In Table 1, we provide a consolidated overview of the reviewed survey papers, including their publication dates and the primary concepts they addressed.

This section discusses currently available research on software vulnerability detection. Several strategies have been proposed and intensively researched in the field of software engineering for discovering shortcomings and vulnerabilities in source code. Early research concentrated mostly on static and dynamic analysis techniques that employed manually generated rules, symbolic execution, or testing frameworks to detect security issues. Over time, the research community has shifted toward more data-driven methods, incorporating pattern recognition, machine learning, and deep learning models to automate vulnerability discovery. Shahriar et al. [24] explored several code analysis techniques, classifying current methodologies according to their methodological focus and detection scope. Malhotra et al. [25] performed a study of software defect prediction approaches that used ML techniques, highlighting their benefits and limitations in detecting fault-prone code sections. In a more comprehensive assessment, Liu et al. [26] provided a broad overview of vulnerability detection research, focusing on works that used both traditional code analysis and ML-based algorithms. Ghaffarian et al. [21] conducted an in-depth review of vulnerability detection utilizing classical machine learning approaches, highlighting features based on software metrics, expert-defined patterns, and anomaly detection. Following that, another review [27] provided a succinct overview of related findings. From a different perspec-

tive, [28] performed a thorough review that identified parallels between programming languages and natural languages and explained how such comparisons influence model design. More recently, several surveys have concentrated on deep learning-based vulnerability detection. Uddin et al. [29] introduced the “Vulnerability Detection Lifecycle” framework to systematically analyze deep learning models across stages like data construction, code representation, and deployment. Hanif et al. [30] executed a comprehensive literature evaluation of 90 materials published between 2011 and 2020, providing two different taxonomies for classifying vulnerability detection studies. Harzevili et al. recently published a thorough systematic literature analysis, providing wide insights on the use of machine learning for software vulnerability detection across multiple data types including source code, binaries, and commit metadata—over a long period of time (2011–2024) [31]. Zeng et al. [32] executed their survey based on four recent “game changer” endeavors. While their methodology recognizes crucial research areas coming from these “game changers”. Lomio et al. [33] evaluated the effectiveness of an existing ML-based SVD method for commit-level detection. However, their selection of just 9 domains seems ambiguous. Liu et al. [34] conducted a survey that compared and analyzed strategies for identifying XSS vulnerabilities. Chakraborty et al. [35] carried out an empirical investigation to assess the robustness of DL-based vulnerability detection approaches when applied to real-world datasets. Their evaluation revealed a critical limitation: although existing DL-based methods often demonstrate strong performance in controlled or benchmarked environments, their effectiveness tends to deteriorate significantly in real-world scenarios. This performance gap highlights the challenges of generalizability, where models trained on curated or synthetic datasets fail to capture the complexity, diversity, and noise inherent in actual software projects. The study underscores the importance of evaluating vulnerability detection methods not only on benchmark datasets but also across diverse, real-world software repositories, thereby ensuring that proposed techniques remain reliable and scalable in practice. Zheng et al. [36] conducted an experiment to determine the impact of various ML methodologies on vulnerability detection performance. Their findings demonstrate that the attention mechanism improves vulnerability identification by allowing models to capture fine-grained contextual relationships in source code. In contrast, the use of transfer learning did not result in significant gains in model performance. This result implies that while transfer learning has been effective in other domains such as natural language processing (NLP) and computer vision, its benefits are less evident in software vulnerability detection, possibly due to the domain-specific characteristics of programming languages and security flaws. In recent times, Zhao et al. [37] proposed Yama, utilizes PHP opcode principles to perform context and path-sensitive intraprocedural analysis. It incorporates accurate type inference, dynamic language structures, and concrete execution for executing challenging built-in functions. This study demonstrates the importance of opcode-level semantic reasoning in enhancing static vulnerability detection precision in dynamic web languages. In another study. In another, Ji et al. [38] introduced Artemis, integrates GPT-4-based source/sink identification, implicit call graph construction, rule-driven taint propagation, and path-condition-based false positive pruning. However, In Table 1, we provide a consolidated overview of the reviewed survey papers, including their publication dates and the primary concepts they addressed.

Previous survey studies have mainly focused on traditional ML- and DL-based methods for vulnerability detection, often providing descriptive overviews without integrating recent developments in LLMs, prompt engineering, or neuro-symbolic reasoning. In contrast, this study contributes a unified and forward-looking synthesis that bridges these paradigms, offering a taxonomy linking datasets, model architectures, and evaluation metrics. Rather than conducting empirical experiments, the emphasis of this survey lies in

conceptual integration, trend analysis, and identification of research gaps to guide future empirical and theoretical advancements in automated vulnerability detection.

Table 1. Overview of the existing review papers.

Published Paper	Published Date	Contributions
Shahriar et al. [24]	2012	Vulnerability detection experiments employing various code analysis approaches.
Malhotra et al. [25]	2015	Programming defects identification methods using ML strategies,
Liu et al. [26]	2012	A comprehensive summary of finding vulnerabilities analysis.
Ghaffarian et al. [21]	2017	Presented fundamental labeling for work in the software vulnerabilities detection area.
Uddin et al. [29]	2025	Deep learning models with code representation, and implementation.
Lomio et al. [33]	2022	The study explored the extent to which Machine Learning has been adopted as an assistive tool for developers in vulnerability detection. By leveraging statistical models and pattern-recognition algorithms, ML enables automated identification of code anomalies, thereby reducing developers' reliance on manual inspection and traditional static analysis methods
Hanif et al. [30]	2021	Designed two alternative taxonomies for software vulnerability detection, one representing research interests and the other highlighting methodological methods.
Harzevili et al. [31]	2024	Explored ML-based methods to analyze publication trends, dataset usage, feature representations, and model architectures, while also identifying the most frequently studied vulnerability types.
Zeng et al. [32]	2020	Highlighted and critically analyzed four seminal papers in the field, emphasizing their transformative impact on the direction of software vulnerability detection research.
Liu et al. [34]	2019	Provided a comprehensive discussion on the classification of Cross-Site Scripting (XSS) attacks, highlighting their types and implications.
Chakraborty et al. [35]	2022	Conducted a survey to evaluate the effectiveness of existing deep learning-based vulnerability detection methods when applied to real-world datasets.
Zheng et al. [36]	2021	In their study, the authors examined how various ML strategies affect the performance of source-code vulnerability detection. By comparing multiple approaches including feature-engineering based models, attention mechanisms, and transfer learning—they evaluated the relative contribution of each strategy toward identifying vulnerable code segments.
Zhao et al. [37]	2025	Provided Yama tools for PHP vulnerabilities identification, where experimental findings on 24 real-world applications (10M LOC) demonstrate a 99.1% detection accuracy and the identification of 38 zero-day vulnerabilities, higher than 9 modern static analyzers.
Ji et al. [38]	2025	Evaluated on 250 PHP applications, it achieved a 90.2% reduction in false positives and discovered 35 new SSRF vulnerabilities (24 CVEs). The framework highlights the potential of combining large language models with program analysis for vulnerability detection

3. Background

3.1. Methodology

PRISMA-ScR Reporting Statement: This scoping review was conducted following the PRISMA-ScR (Preferred Reporting Items for Systematic Reviews and Meta-Analyses extension for Scoping Reviews) guidelines to ensure transparent and comprehensive reporting of the methodology. The literature selection and screening process, including the identification, exclusion of duplicates, abstract-level screening, and final inclusion of 169 studies, is summarized in Figure 2.

In this work, we employ a methodical review procedure that utilizes findings from three major sources: software vulnerability-focused review papers, experimental research studies, and empirical surveys. Our major goal is to provide future researchers with a thorough grasp of the code patterns, models that have been commonly employed in recent years, and types of vulnerabilities discovered. We begin by considering a variety of review papers which describe historical and present techniques to vulnerability identification. It helps us to observe how older technologies, such as rule-based static analysis and

classical ML, have been gradually succeeded or complemented by DL and LLM-based approaches. Recent studies demonstrate a progressive movement of completely human-centered feature engineering and toward data-driven techniques that automatically learn feature representations from source code semantics. Rather than upgrading previous static and rule-based methods, advanced ML, DL, and LLM-based methodologies have been employed as supplemental solutions to improve scalability, automation, and semantic understanding in vulnerability identification. After consolidating findings from these three categories of literature, we synthesize and discuss the challenges and limitations of the existing body of work. These challenges include issues such as dataset imbalance, false positive rates, limited generalizability across programming languages, and the scarcity of benchmark datasets particularly for languages like C/C++, Python and Java. Finally, we identify future research directions that can bridge the gaps in current studies. These future scopes include the integration of con-text-driven learning, hybrid symbolic–neural approaches, improved dataset construction pipelines, and explainable AI techniques for vulnerability detection. The workflow of our methodology is illustrated in Figure 3, which demonstrates the overall process from literature selection to the identification of future research opportunities.

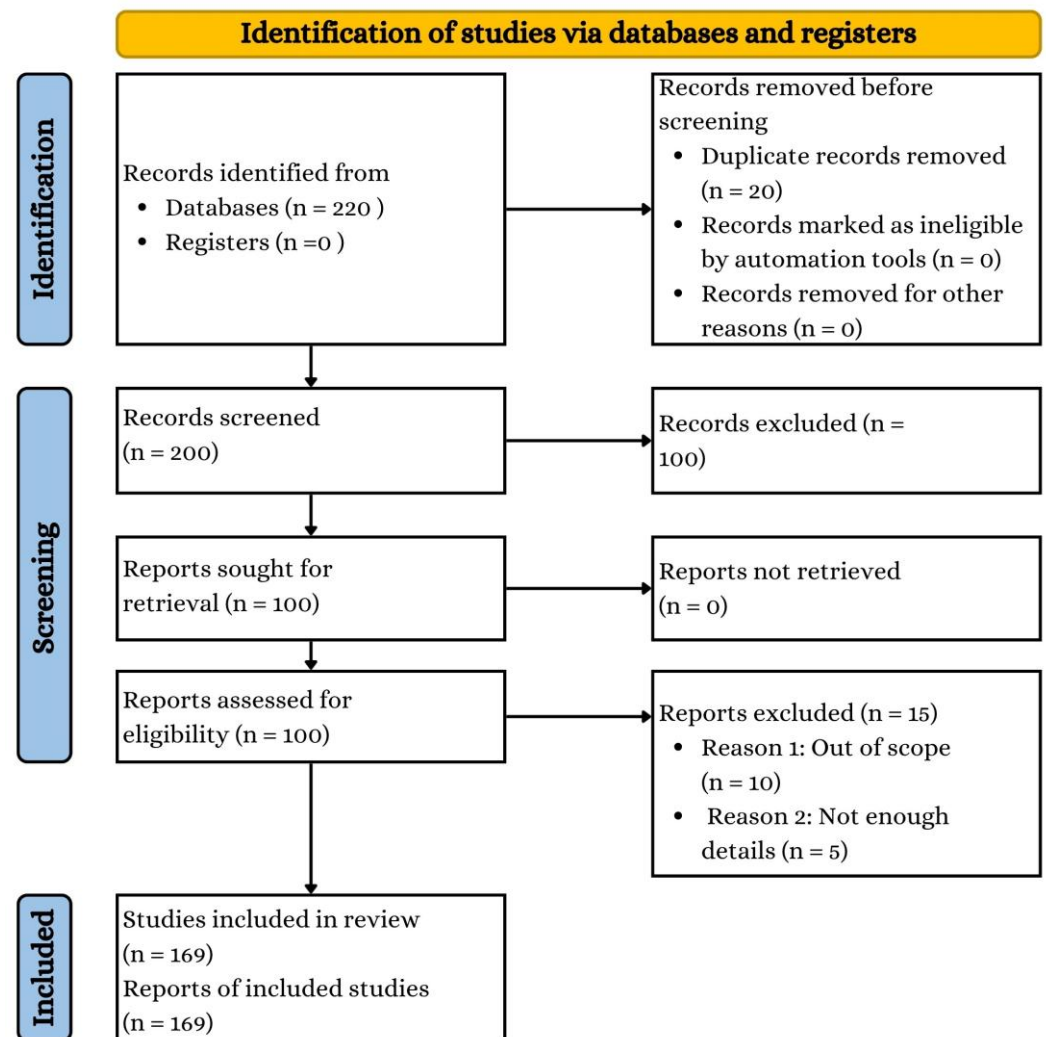


Figure 2. PRISMA-ScR flow diagram illustrating the literature identification, screening, and inclusion process.

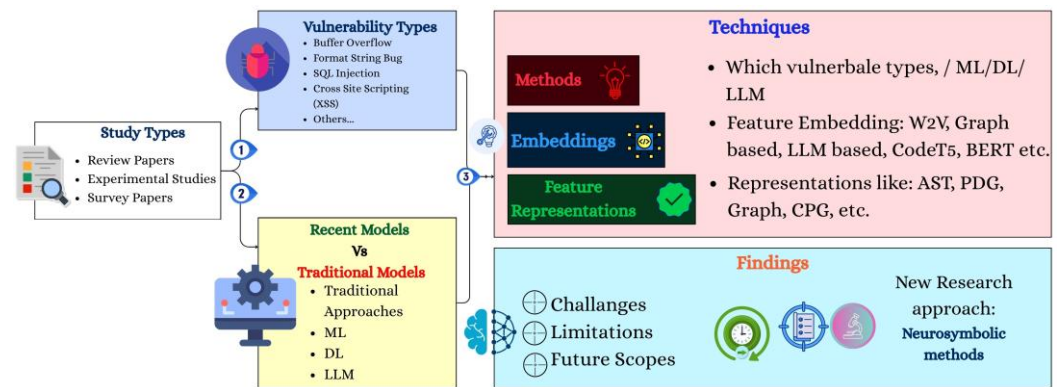


Figure 3. An overall overview of the study. The figure illustrates the structured workflow, starting with the review of existing literature (review papers, experimental studies, and survey works), followed by an analysis of vulnerability types, traditional versus modern detection models, and state-of-the-art (SOTA) methods. The synthesized findings are then critically examined to identify challenges and limitations, which ultimately highlight future research directions.

3.2. Vulnerability Types

Modern software systems have grown more interconnected, data-driven, and developed from complex dependency relationships. While this environment drives advancement, it also increases the attack surface and raises the implications of minor code errors. As a result, vulnerability identification has evolved from chaotic finding bugs to methodical, data-driven methodologies that rely on program analysis and artificial intelligence. One of the most common vulnerable types is Memory-safety errors (stack/heap) buffer overflows (CWE-121/122) and generalized insufficient accesses (CWE-787) arise when updates and reads exceed an object's bounds, typically due to unchecked copying, faulty index arithmetic, and size calculation. The implications include data corruption and control-flow disruption. In another, use-after-free, double-free, and dangling pointers, where lifetime mismanagement leads to use-after-free (CWE-416), double-free (CWE-415), and dangling pointer dereferencing. These flaws commonly provide arbitrary memory writes. Sound interprocedurally escape analysis remains difficult; sanitizers catch many situations at runtime but have coverage constraints. Integer overflow/underflow (CWE-190), width conversion errors, and size-calculation issues spread through allocator calls, resulting in undersized buffers and wrapped indices that cause memory violations. Then, Uncontrolled format strings (CWE-134) occur when attacker-controlled strings are used as format specifiers like `printf (user)`, enabling information disclosure or memory writes (`%n`). Reads of uninitialized variables (CWE-457), improper pointer provenance, alignment violations, and other types of undefined behavior reduce program correctness and might result in security or dependability flaws. Race conditions (CWE-362), TOCTOU file races (CWE-367), deadlocks, and atomicity violations are caused from poor synchronization across shared state or time-of-check/time-of-use gaps. Because these flaws are schedule-dependent, pure fuzzing struggles with nondeterminism. In another, Leaks of memory or file descriptors (CWE-772), improper cleanup, and exception-unsafe resource handling reduce availability and create side channels. Improper privilege management (CWE-269), insecure search paths and load-order spoofing, unsafe temporary files, and capability misconfiguration could lead to escalation or arbitrary code execution in native systems. Insecure compiler/linker flags, dangerous defines, or brittle installer logic can re-enable undefined behavior or bypass mitigations such as disabling stack canaries. Practical detection integrates static policy checks in CI with learned heuristics over build manifests and scripts. Use of broken primitives, weak PRNGs (CWE-330), incorrect modes hard-coded keys, or missing salts undermines confidentiality and integrity. These are best identified by API-misuse checks enriched with

semantic reasoning; few-shot prompts over code + API docs can guide LLMs to recognize insecure cryptographic patterns. However, Figure 3 illustrates the overall distribution of vulnerability types and corresponding C/C++ code samples.

According to Figure 4, SQL Injection (SQLi) in Python (version 3.10 & 3.9) is shown where user input is directly concatenated into a SQL query, allowing attackers to manipulate database commands. Similarly, OS Command Injection (OSCi) in Python demonstrates the danger of passing unvalidated input to system commands, enabling remote code execution. Cross-Site Scripting (XSS) in Java is illustrated where unescaped user input is reflected in a webpage, leading to malicious script execution in the browser. Path Traversal in Python highlights how unsafe file path handling allows attackers to access restricted files such as /etc/passwd. Insecure Deserialization (ID) in Python is shown with pickle.loads, where malicious payloads can execute arbitrary code. Server-Side Request Forgery (SSRF) in Java demonstrates how unvalidated URLs may allow attackers to access internal resources like AWS metadata services. Finally, Buffer Overflow in C/C++ is presented with an example of writing beyond the bounds of a buffer, a classic memory safety flaw that can lead to program crashes or arbitrary code execution. In Table 2, we provide an overview of the most prevalent and practically dangerous vulnerability types, emphasizing their frequency in real-world systems and the significant security risks they pose.

Memory Safety (buffer overflows, OOB) <pre>1.char buf[8]; 2.strcpy(buf, input); 3. // overflow if input ≥ 8 4.// safer: strncpy(buf, input, sizeof(buf)-1); buf[7] = '\0'; 1.int a[4]; 2.a[5] = 1; 3.// out-of-bounds write 4.// safer: check index < 4</pre>	Integer and Size Errors <pre>1.size_t n = get_n(); 2.size_t bytes = n * sizeof(struct S); 3. // may overflow 4.void *p = malloc(bytes); 5. // under-sized allocation 6.// safer: check n ≤ SIZE_MAX/sizeof(struct S) before multiply</pre>	Concurrency Defects (races, TOCTOU, deadlocks) <pre>1.// data race 2.shared++; 3.// written without lock in multiple threads 4.// safer: protect with mutex / atomic 1.// TOCTOU 2.if (access(path, W_OK) == 0) fd = open(path, O_WRONLY); 3.// safer: open() with O_CREAT O_EXCL and proper permissions 1.// deadlock (lock order inversion) 2.lock(m1); lock(m2); 3.// another thread uses m2→m1 4.// safer: enforce a global lock ordering</pre>
Format-string Issues <pre>1.char *user = argv[1]; 2.printf(user); 3. // uncontrolled format string 4.// safer: printf("%s", user);</pre>	Uninitialized / Undefined Behavior <pre>1.int x; if (x) puts("hi"); 2.// read of uninitialized x 3.// safer: int x = 0;</pre>	API / Contract Misuse <pre>1.char *p = malloc(16); 2.memcpy(p, src, 64); // violates size precondition 3.// safer: memcpy(p, src, 16); or validate length</pre>
		Privilege / Environment Misuse <pre>1.char cmd[64]; sprintf(cmd, "/bin/cp %s %s", a,b); system(cmd); 2.// safer: use execv with fixed arguments; avoid shell</pre>

Figure 4. Examples of Vulnerability types based on C/C++, Java, and Python code.

Table 2. Overview of the most prevalent and practically dangerous vulnerability types.

Vulnerability Type	Most Common/Dangerous	Real-World Impact & Statistics	References
SQL Injection (SQLi)	Massive data exposure through unsanitized query inputs. Widely exploited across industries.	2023 ResumeLooters campaign hit 65+ websites, stealing over 2M records. Equifax 2017 breach traced to SQLi.	[39,40]
Cross-Site Scripting (XSS)	Found in nearly one-third of web apps; enables session theft and phishing via injected scripts.	XSS present in 19.1% of web apps with medium severity in 2023 reporting.	[41,42]

Table 2. *Cont.*

Vulnerability Type	Most Common/Dangerous	Real-World Impact & Statistics	References
Buffer Overflow	Continues to dominate the 2024 CWE Top 25 due to memory corruption risks in unmanaged languages.	The SQL Slammer worm (2003) exploited such a bug to cause global outages within minutes.	[43]
SSRF (Server-Side Request Forgery)	Prolific in cloud environments; attackers exploit internal service access.	Ranked significant in the 2023 Vulnerability Report; cloud metadata targeting is notable.	[41]
Path Traversal	Allows attackers to access restricted files via malformed path input.	Jira CVE-2019-11581 enabled path traversal to expose sensitive files.	[40,43]
Insecure Deserialization	Deserialized untrusted data can execute arbitrary code; critical risk in Python/Java ecosystems.	Apache Commons Collections exploit (2015) is a prime example used in high-profile breaches.	[44]
XML External Entity (XXE)	Misconfigured XML parsers allow file disclosure or SSRF.	Still recurring in enterprise environments due to insecure defaults and legacy systems.	[45]
OS Command Injection	Injected shell commands compromise host; extremely powerful if exploited.	Log4Shell (2021) indirectly leveraged command injection vectors; Shellshock (2014) is another critical case.	[44]

3.3. Traditional Methods

3.3.1. Static Analysis

Static analysis refers to the examination of a software system or its components by analyzing their structure, syntax, and documentation, without the need to execute the program [46]. The main idea of static analysis is to analyze source code or program language without executing it in order to uncover errors, weaknesses, or trends that may lead to software vulnerabilities. Traditional static analysis methods include manual code inspection, in which human analysts thoroughly inquiry the source code to find shortcomings. While effective in some situations, this procedure is resource-intensive, time-consuming, and heavily reliant on inspectors' skill and prior knowledge of known vulnerabilities. According to Rice's Theorem [47], any non-trivial semantic property of programs is undecidable, meaning it cannot be determined by any general algorithm. This result is a direct consequence of the halting problem, as Rice demonstrated that determining whether a program possesses a particular can be reduced to solving the halting problem itself. Another basic behavior of static analysis is its conservatism. Because static analysis must function without executing the program, it usually errs on the side of caution, listing all possible behaviors that may occur. While this conservative technique ensures that potential vulnerabilities are not discovered, the results are overly approximate and, in certain cases, imprecise [48]. To overcome these limitations, researchers have developed a wide range of static analysis tools that automate parts of the process by applying predefined rules, heuristics, or formal methods to detect security flaws. Rule-based static analysis is based on preset vulnerability patterns or coding standards obtained from secure programming guidelines. This technique uses tools to check source code against a library of criteria such as "avoid unsafe functions like strcpy" or "sanitize user inputs before SQL queries." The main advantage of rule-based systems is their interpretability, which allows engineers to readily understand why a warning was sent. However, these solutions are fragile, frequently failing to generalize to new vulnerabilities outside of their specified rule set. Furthermore, they could deliver false positives if the criteria are incredibly broad, or false negatives if the vulnerability does not perfectly fit the pattern. FindBugs [49], and Flawfinder [50] for C/C++ rely heavily on rule-based detection. With the rise of large-scale code repositories, researchers have explored

detecting vulnerabilities by comparing code fragments for similarity [51,52]. The intuition is that vulnerable code often reappears across projects due to copy-paste programming or repeated insecure practices. Code similarity techniques use program representation models. Symbolic execution is a path-sensitive analysis approach that runs a program using symbolic inputs rather than physical values. It carefully investigates potential execution routes by solving symbolic constraints, revealing vulnerabilities like buffer overflows, path traversal, and assertion violations. Symbolic execution is very precise and may even detect zero-day vulnerabilities, but it suffers from the well-known route explosion problem, in which the number of alternative pathways increases exponentially with program size [53]. Unlike basic pattern matching, nowadays static analysis propagates facts over the CFG/ICFG to identify the least fixpoints for data-flow characteristics. Taint analysis is central, monitoring flows from sources through propagators to sinks while accounting for sanitizers, aliases, and procedure calls; a warning is delivered when a contaminated value can reach a sink over a potential path. Complementary components include abstract interpretation (ranges/nullness/size), point-to/alias reasoning for heap updates and lifetimes (use-after-free/double-free), and concurrent models (races/TOCTOU). We examine the merits (explainable source-sink traces) and limits (over-approximation, framework modeling) of ML/LLMs, positioning them as complements that rank and summarize, while symbolic execution/fuzzing validates feasibility.

3.3.2. Dynamic Approach

Unlike static analysis, which analyzes source or binary code prior to execution, dynamic analysis reviews a program while it is running. This helps researchers and security practitioners to identify vulnerabilities that appear only under certain runtime situations, such as memory corruption, improper input processing, or unexpected program states. Fuzzing and taint analysis are two of the most often used approaches. Fuzz testing (fuzzing) is an automated software testing technique where a program is bombarded with large volumes of random, malformed, or crafted inputs to trigger unexpected behavior. The core idea is that vulnerabilities such as buffer overflows, use-after-free, and input validation flaws often surface when the program processes unusual inputs. Miller et al. [54] developed the “fuzz” program in 1990, inspired by the failure of modem systems given the unpredictable input from “fuzzy” lines. The program creates flows of letter sequences. Building on this perspective, Oehlert [55] described fuzzing as a highly automated testing technique capable of methodically covering border instances. Fuzzing, according to this description, is the process of generating flawed or unexpected inputs from various sources, such as files, network protocols, and API requests, in order to find exploitable holes. Similarly, Sutton et al. [17] classified fuzzing as a kind of brute-force testing, emphasizing its emphasis on exhaustively feeding huge numbers of test cases to the application under test. To supplement these viewpoints, Lanzi et al. [56] defined fuzzing as a sort of black-box testing in which the program’s underlying logic remains opaque and the focus is primarily on monitoring input-output behaviors. Taint analysis dynamically tracks how untrusted inputs (tainted data) propagate through a program and determines whether they reach sensitivity without proper sanitization. The concept is that any data originating from an external user should be considered tainted, and its use in security-critical operations must be validated [18,57]. Hybrid approaches that combine symbolic execution with heuristics, fuzzing, or machine learning intent for solving this scaling issue.

3.4. Code Based Detection

3.4.1. ML Based Methods

Early machine learning (ML)-based approaches to software vulnerability detection frequently relied on software metrics as indicators of code quality and risk. These methods aimed to capture structural and complexity-related features of source code and use them as input for classifiers that distinguish vulnerable from non-vulnerable functions. For example, Zagane et al. [58] employed metrics such as total lines of code, cyclomatic complexity, and the number of distinct operators to quantify software artifacts at a finer granularity. This quantification enabled them to characterize code fragments and evaluate their likelihood of containing vulnerabilities using classifiers like RF, DT, and K-Nearest Neighbors (KNN). Such work demonstrated that code complexity and operator diversity may be strong indicators of potential flaws. Building upon these efforts, Salimi et al. [59] introduced the concept of vulnerable slices, which represent critical code segments that are particularly prone to defects. This slicing approach allowed for more targeted feature extraction and improved the precision of ML models by narrowing the focus to code regions most relevant to security analysis. Collectively, these studies underscore the potential of metric-based approaches for vulnerability detection, while also highlighting their limitations in handling semantic and contextual aspects of modern software systems. Several studies have investigated the use of machine learning techniques to predict when and how software components should undergo refactoring. Kosker et al. [60] trained a Naïve Bayes (NB) classifier to identify classes in object-oriented systems that required refactoring. Their work demonstrated that lightweight probabilistic models could provide actionable insights into maintainability improvements. Expanding on this line of study, Kumar and Sureka [61] used Least Squares-Support Vector Machine (LS-SVM) classifiers modified with the Synthetic Minority Oversampling Technique (SMOTE) to solve class imbalance. Their findings demonstrated that the LS-SVM model with a Radial Basis Function (RBF) kernel outperformed other configurations, giving it a promising contender for practical refactoring prediction. In another study, Nyamawe et al. [62] proposed a more comprehensive framework that leveraged both software evolution history and prior refactoring actions. Their method was structured as a two-stage classification pipeline: (1) a binary classification task to determine whether a refactoring was needed, and (2) a multi-label classification task to recommend the specific type of refactoring to apply. Figure 5, represent of traditional ML models employed for software vulnerability detection across existing studies (2015–2025). Among these, RF and SVM are the most widely adopted models.

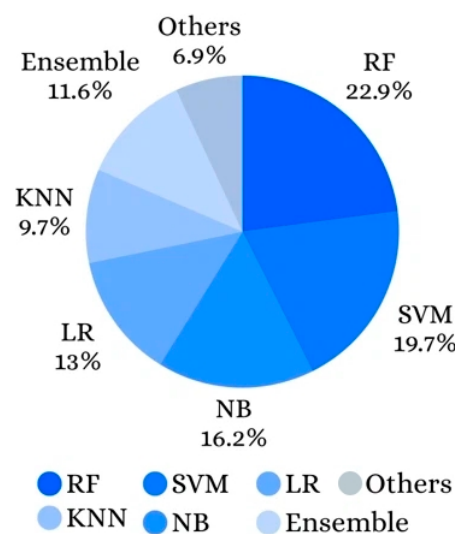


Figure 5. Distribution of traditional machine learning models applied in software vulnerability detection.

3.4.2. DL Based Methods

With the increasing success of DL in fields such as computer vision and natural language processing, the software vulnerability detection community began to transition toward DL-based approaches. These methods eliminated the need for manually engineered features by enabling neural networks to automatically learn semantic and syntactic representations of code. Recurrent Neural Networks (RNNs) and their variants have been widely adopted in software vulnerability detection due to their ability to model sequential dependencies in code. Among these, Long Short-Term Memory (LSTM) networks have emerged as one of the most prominent techniques. LSTMs effectively capture long-term contextual relationships in sequential data, mitigating the vanishing gradient problem faced by basic RNNs. Several studies have successfully applied LSTMs in source code vulnerability detection, demonstrating their robustness in modeling code semantics and flow [63–68]. The Bidirectional LSTM (BiLSTM), an extension of this model, improves efficiency by analyzing input sequences in both forward and backward directions, enabling for more context-aware representations of source code. As a result, BiLSTMs have been widely used in various vulnerability detection activities [15,69–73].

Beyond LSTM-based models, researchers have also explored other recurrent architectures for software vulnerability detection. The Bidirectional RNN (BiRNN) extends the standard RNN by processing sequences in both forward and backward directions, thereby capturing richer contextual information from source code. Despite its potential, this model has seen very limited application in the literature, with only a single study reporting its use [74]. The Gated Recurrent Unit (GRU) has emerged as a simplified alternative to LSTM, designed to address the same long-term dependency challenges while offering reduced computational complexity. This trade-off makes GRUs particularly appealing for large-scale vulnerability detection tasks where training efficiency is critical. Nevertheless, GRUs have been adopted sparingly, with only two studies leveraging this architecture for vulnerability-related analysis [67]. An extension of this model, the Bidirectional GRU (Bi-GRU), processes sequences in both directions to enrich contextual learning. By doing so, it provides a more comprehensive understanding of source code semantics. This architecture has been evaluated in two additional studies, highlighting its niche but growing role in vulnerability detection research [65,75]. Finally, more advanced generative approaches have also been explored. For example, SeqGAN, a Generative Adversarial Network (GAN) designed specifically for sequential data, has been applied in one study [33].

Graph Neural Networks (GNNs) are a class of models designed to detect relationships and relational structures in graph-structured data. They are especially valuable in software vulnerability identification since source code may be readily expressed as graphs like Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), and Program Dependence Graphs. These structures store both syntactic and semantic information, allowing GNNs to accurately represent data flow and control inside a program. Most of the researchers employed basic GNNs for vulnerability detection, demonstrating their ability to detect subtle code weaknesses by leveraging structural dependencies [76–83]. A notable extension of this framework is the Gated Graph Sequence Neural Network (GGNN), which incorporates gating mechanisms to capture sequential dependencies within graph representations. This architecture enables the model to track how information propagates across program elements over time, thereby improving its ability to detect vulnerabilities arising from complex code interactions. Existing studies applied GGNNs in this context, where the gating mechanism enhanced feature representation and improved detection accuracy [84–88]. The Relational Graph Convolutional Network (RGCN) is designed to accommodate a variety of node connections. In software engineering, this is equivalent to modeling various interactions such as data dependencies, control dependencies, and API call interactions. By distinguishing between

various relationship types, RGCNs deliver a more comprehensive description of program structure as found from the existing studies [89,90]. The Graph Convolutional Network (GCN) is a simplified version of GNNs that focuses on efficient convolutional operations on graph topologies. GCNs work effectively for applications that need only local connection and neighborhood patterns, such as finding clusters of susceptible services or modules. We discovered some studies that used GCNs, particularly in static vulnerability detection environments, and shown its efficacy for finding vulnerabilities at scale [83,84,91]. Convolutional Neural Networks (CNNs) represent one of the earliest and most widely adopted deep learning architectures applied to software vulnerability detection. Originally developed for image recognition, CNNs have demonstrated strong capabilities in scanning structured and sequential data to automatically learn discriminative patterns. In the context of source code analysis, CNNs treat code tokens, embeddings, or program slices as input sequences or matrices, enabling them to extract hierarchical features that differentiate between vulnerable and non-vulnerable components. Their effectiveness lies in the ability to capture both local and global dependencies in code, which are essential for recognizing subtle patterns of insecure constructs [92]. CNN-based techniques have been widely used in a variety of research [65,93–99]. These studies show that CNNs are particularly efficient in detecting vulnerabilities such as buffer overflows, SQL injection, and API exploitation, where syntactic and semantic regularities might be acquired from token sequences. Furthermore, CNNs are frequently used as baselines for comparison with more complex deep learning models because of their efficiency and affordable computational cost. In the field of vulnerability detection, DGCNNs have been employed to identify complex vulnerability patterns that cannot be easily detected through linear code representations. For instance, one notable work [100] applied DGCNNs to model the relational and structural aspects of source code, thereby improving the detection of vulnerabilities associated with data flow and control flow interactions. Compared to traditional CNNs, DGCNNs provide a richer representation by considering the hierarchical and relational nature of source code, which is particularly useful for detecting vulnerabilities that emerge from non-trivial execution paths and contextual interactions.

Figure 6 provides a comprehensive overview of the deep learning models that have been most frequently employed in software vulnerability detection research. Overall, the figure highlights a clear research trend: sequence-based models (CNN, LSTM, BiLSTM) have been the most frequently employed approaches for vulnerability detection, underscoring the importance of analyzing code as sequential data. Meanwhile, graph-based approaches (GNN and its variants), though less common, represent an emerging research direction as they offer unique advantages in capturing the structural and relational properties of source code. This comparison suggests that while traditional sequential models remain dominant, graph-based models are steadily gaining traction in vulnerability detection studies.

3.4.3. Large Language Models (LLMs)

Large Language Models (LLMs) represent a significant step forward in the development of language modeling approaches [101,102]. Figure 7 illustrates the recent utilization of LLMs across various applications. Building on the Transformer architecture, these models have shown the ability to expand to unprecedented levels, frequently comprising hundreds of billions of parameters. Furthermore, training LLMs on vast and varied structures has provided them with a level of generality not possible with previous generations of models. This generalization capacity implies LLMs to adapt to new domains and tasks with minimum fine-tuning, emphasizing their importance as foundation models in current AI research [103]. Their scalability also highlights a key trend in deep learning: performance

improves as model size, data, and compute resources grow; a phenomenon known as “scaling laws” [104,105]. Pan et al. [106] proposed a taxonomy that categorizes LLMs into three primary architectural groups: encoder-only, encoder–decoder, and decoder-only models.

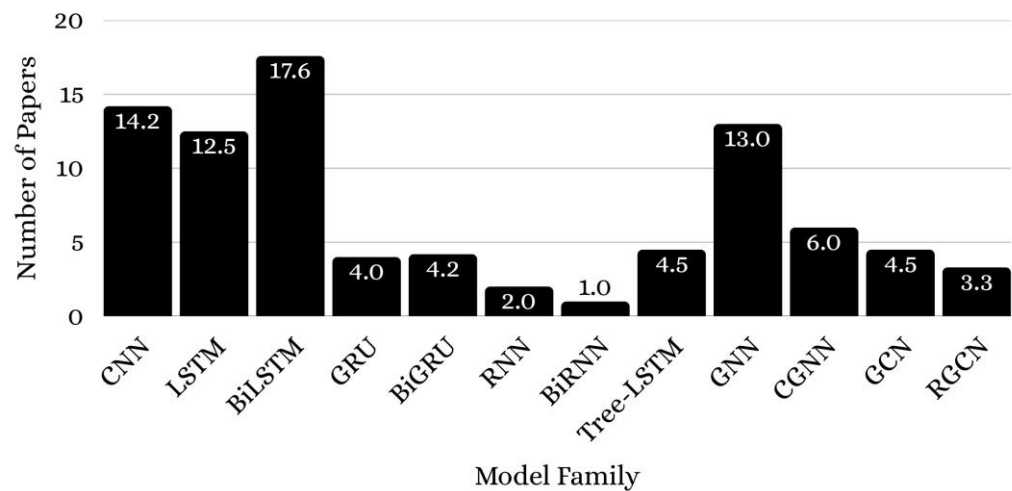


Figure 6. Deep-learning based vulnerability detection models from 2015–2025, where mostly used BiLSTM, CNN, LSTM, and GNN model mostly.

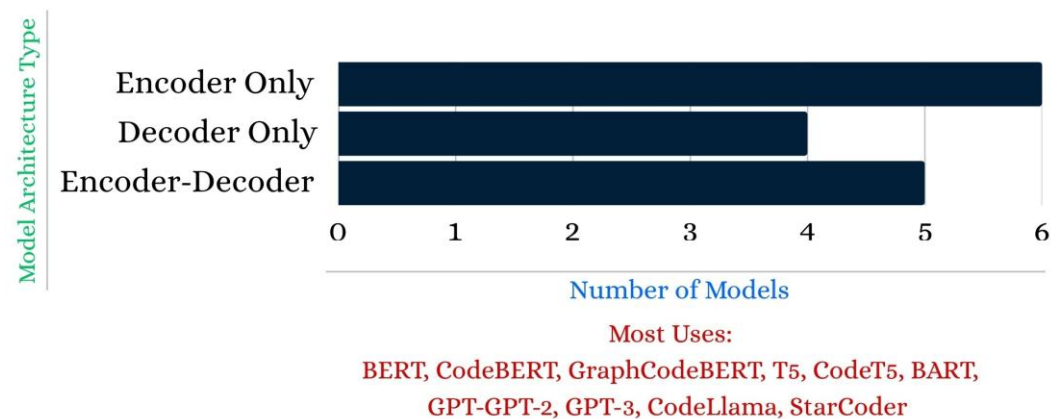


Figure 7. Overall comparison of recently employed LLM.

Encoder-Only. Encoder-only models depend based only on representation learning. These models, notably BERT [107], are designed to capture deep bidirectional contextual embeddings, making them ideal for classification, feature extraction, and semantic similarity tasks. Their strength is in creating rich contextual representations of input sequences, that could further be used in downstream supervised learning problems. In order so, these mathematical representations provide fine-grained analysis of programming structures, making them extremely useful in software engineering (SE) research. Several customized encoder-only models have been developed for SE purposes. In particular, CodeBERT [108] and GraphCodeBERT [109] expand BERT-like structures to source code and data-flow graphs, respectively, whilst CuBERT [110] focuses on large-scale pretraining of Python programs. Other domain-adapted encoder models include VulBERTa [111] for vulnerability detection, CCBERT [112] for code comprehension. For vulnerability detection most of the study used BERT and CodeBERT most for the evaluation and their performances [113–116].

Encoder–Decoder. Encoder–decoder models leverage the combined strengths of both the encoder and decoder components of the Transformer architecture. The encoder processes the input sequence and transforms it into a high-dimensional structured representation, which captures both syntactic and semantic properties. The decoder then

generates the corresponding output sequence conditioned on this representation. This dual structure provides high versatility, making encoder–decoder models particularly effective for tasks such as translation, summarization, defect repair, and code transformation. For instance, models like PLBART [117] and T5 [118] were originally proposed for natural language processing (NLP) but have been successfully adapted to SE tasks due to their ability to learn cross-domain semantics. Similarly, CodeT5 [119] extends the T5 architecture for programming languages, demonstrating strong performance in code summarization and vulnerability detection. Other representative models include UniXcoder [120], which unifies programming language and natural language tasks in a single framework, and NatGen [121], which is designed for code generation and related SE and SVD tasks.

Decoder-Only. Decoder-only LLMs are built solely on the decoder component of the Transformer architecture, making them inherently generative in nature. These models are designed to produce text or source code sequences conditioned on an input prompt. By relying on autoregressive token prediction, they interpret contextual information from the prompt and iteratively generate subsequent tokens. This design enables decoder-only models to construct coherent, contextually relevant, and semantically rich sequences, which makes them particularly effective for tasks such as dialogue systems, text completion, and code generation.

Their strength lies in the ability to extend and elaborate context dynamically, producing complex outputs that align with both local syntax and global semantics of the input. Significant models in the decoder-only LLMs category include the GPT family, which includes GPT-2 [122], GPT-3 [123], GPT-3.5 [124], and the most current GPT-4 [125]. In addition to general-purpose models, some architectures have been designed specifically for source code and software engineering applications. These include CodeGPT, Codex, Polycoder, Incoder, the CodeGen series, as well as specialized systems like Code Llama, and StarCoder [126–134]. In combination, these models highlight the rapid growth of decoder-only LLMs from wide natural language production to highly specialized tools for programming and vulnerability identification [135–138].

In Figure 8, illustrates a taxonomy of vulnerability detection approaches, classified into Traditional, ML, DL, and LLMs. Traditional approaches include static (AST, CFG, rule-based) and dynamic (fuzzing, symbolic execution) analysis. ML models rely on handcrafted features with classifiers like SVM and Random Forest. DL models use CNNs, RNNs, GNNs, and Transformer encoders to learn structural and semantic patterns. LLM models leverage prompting, fine-tuning, and hybrid reasoning with advanced code models (GPT-4, CodeLlama, CodeBERT). Each category highlights common techniques, strengths, and weaknesses.

Figure 9 shows the evolution of vulnerability detection research from standard ML to advanced LLM-based approaches. The number shows that LLM/Transformer-based models currently dominate research (31.6%), followed by DL (21.1%), with traditional ML, graph-based, and hybrid techniques contributing approximately 15–16% for each. The related line graph shows a higher trend and increased focus on LLM-driven systems, emphasizing their superior capacity to represent complex structural and conceptual relationships when compared to traditional systems.

3.5. Feature Representation

3.5.1. Graph Based Feature Representation

In this section, we review prior studies that employ different graph-based program representations such as Abstract Syntax Trees (ASTs), Control Flow Graphs (CFGs), Program Dependence Graphs (PDGs), and Data Dependence Graphs (DDGs) as inputs to deep neural networks (DNNs) for learning rich feature representations. In this section, source code is

represented functionally using graphs and trees to encapsulate relationships, control flows, and program semantics. The Control Flow Graph (CFG) is a fundamental representation that describes a program's execution order by depicting the transitions between distinct code segments [59,80,91,113,134–137]. In addition to CFGs, the Program Dependence Graph (PDG) encodes both control and data dependencies between program statements, providing a more comprehensive view of inter-statement relationships [76,79,82,139–141]. Among all structural forms, the Abstract Syntax Tree (AST) remains the most widely adopted, representing syntactic constructs in a hierarchical tree that supports both syntactic and semantic analysis [64,75,79,138]. Variants such as the Control Flow Abstract Syntax Tree (CFAST) enrich ASTs with control flow information for enhanced program representation [139–141], while the Binary AST adapts tree-based structural representations for binary-level code analysis [93].

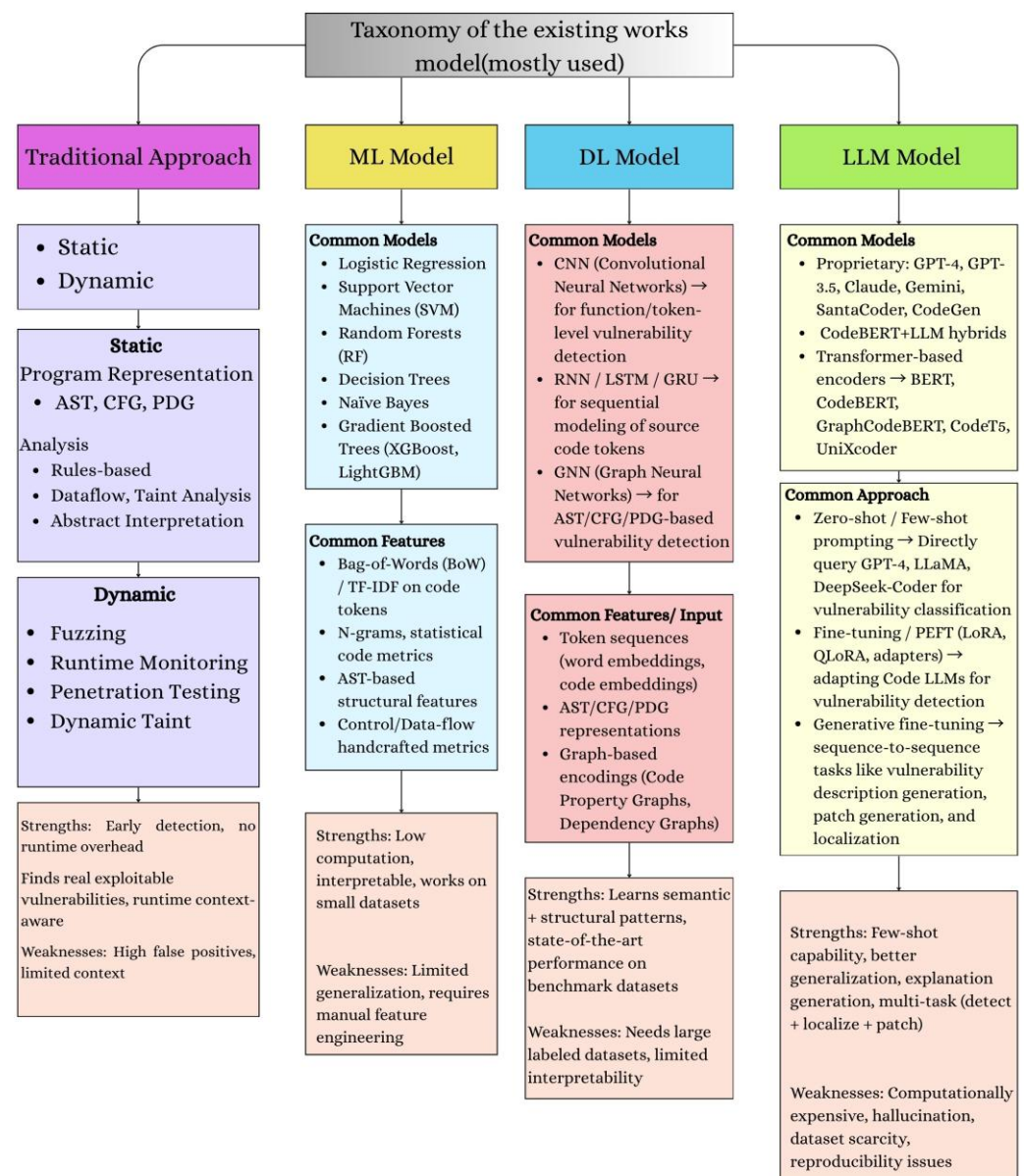


Figure 8. Taxonomy of existing vulnerability detection approaches, categorized into Traditional (static/dynamic analysis), ML-based, DL-based, and LLM-based models, showing their common techniques, features, strengths, and weaknesses.

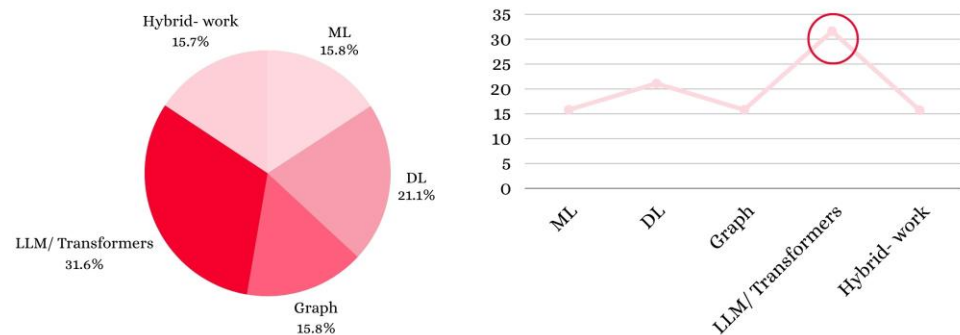


Figure 9. Comparative analysis of research trends in software vulnerability detection using Machine Learning (ML), Deep Learning (DL), Graph-based models, Large Language Models (LLMs/Transformers), and Hybrid approaches. The red circle indicates the highest contribution among categories. Percentages are rounded to one decimal place; total may not be exactly 100% due to rounding.

3.5.2. Text Based Feature Representation

A recent method introduced in [142] transformed Java source files into sequences of textual tokens and utilized an N-gram model to embed these tokens into vector representations. The underlying assumption was that vulnerable patterns could be revealed by analyzing the frequency distributions of code tokens. A different approach CNN-based technique presented in [143] focused on discovering vulnerabilities in C programs at the assembly level, with the goal of directly extracting dangerous code patterns from assembly instructions. To represent these instructions as vectors, the authors proposed Instruction2vec, a model based on the Word2vec framework [144,145]. A analogous CNN-based technique for function-level vulnerability identification was described in [146]. The study made use of a huge dataset of around 12 million source code functions obtained from the Juliet Test Suite, the Debian Linux distribution, and GitHub. To preprocess the code, the authors created a bespoke C/C++ lexer that converted each function into a series of tokens. The lexer limited the vocabulary to 156 tokens, which included all C/C++ keywords, operators, and separators, and removed non-essential code components that had no effect on compilation. Another popular form is the Code Slice, which separates specific chunks of source code based on established slicing criteria such as data and control dependencies. This method allows for the extraction of significant fragments that are most likely to include susceptible patterns and has been used in various research [65,79,147–152]. In contrast, the Code Snippet format concentrates on short, continuous pieces of code, which are usually retrieved for focused study of specific code sections [71]. Other representation and embedding techniques have also been adopted in recent studies, including GloVe, Doc2Vec, Continuous Bag-of-Words (CBoW), Sent2Vec, GRU-based embeddings, FastText, and N-gram models [65,69,70,75,137,147,149,152].

3.6. Prompt Engineering Techniques

Prompt engineering is the process of designing and refining inputs (prompts) given to a LLMs so that it produces more accurate, reliable, or useful outputs. Since LLMs are sensitive to how instructions are phrased, effective prompting can drastically improve performance on tasks like text generation, reasoning, summarization, or vulnerability detection.

Chain-of-Thought (CoT). Chain-of-thought prompting asks the model to provide step-by-step reasoning before giving the final answer. This improves both interpretability and accuracy in reasoning-intensive tasks such as vulnerability analysis [151,152]. According to Figure 10, the model might reason: (1) sprintf copies input without bounds checking, (2) the destination buffer is small (10 bytes), (3) untrusted input may exceed the buffer. Based on

this reasoning, the final classification is 1 (vulnerable). CoT has been shown to improve performance in complex tasks by guiding the model through intermediate reasoning steps rather than relying on a direct prediction.

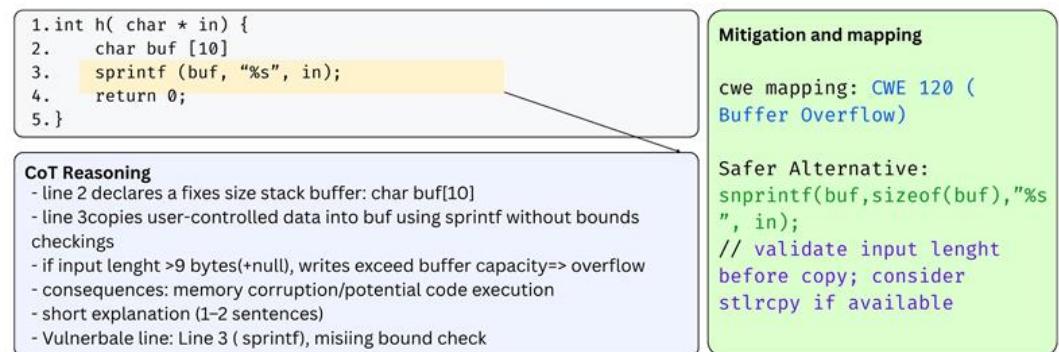


Figure 10. CoT examples based on C/C++ code snippet.

Zero-Shot Prompting. Zero-shot prompting is one of the most basic yet widely used strategies, where the model is asked to perform a task without being provided with any prior examples [153]. The model relies entirely on its pre-trained knowledge to infer the answer from the given instruction. For example, according to Figure 11, the function copies user input into a fixed-size buffer using strcpy, which does not perform bounds checking. This can lead to a stack-based buffer overflow if the input string exceeds 15 characters. To analyze this vulnerability, applied a zero-shot prompt, instructing the model to identify the vulnerable line, classify the vulnerability, assign CWE identifiers, assess severity, and suggest a safer alternative. The model's structured JSON response correctly flagged line 3 as vulnerable, labeled it a buffer overflow, mapped it to CWE-120 and CWE-242, and recommended replacing strcpy with strncpy along with explicit null termination. This illustrates how zero-shot prompting can be effectively used to generate precise, structured vulnerability reports directly from source code without additional training.



Figure 11. Zero-shot prompting examples based on C/C++ code snippet.

Few-Shot Prompting. Few-shot prompting is a technique used in LLMs interactions where the model is given a small number of worked examples (inputs paired with expected outputs) before being asked to analyze a new, unseen instance. Unlike zero-shot prompting where the model must rely solely on instructions, few-shot prompting provides concrete demonstrations of the desired reasoning process and output format [154]. This example in Figure 12, demonstrates the essence of few-shot prompting: the model learns from a small, labeled example (f()) how vulnerabilities are reported. The functions suffer from the same type of bug (a stack-based buffer overflow caused by strcpy), but the buffer sizes differ.



Figure 12. Few-shot prompting examples based on C/C++ code snippet.

3.7. Fine Tuning

Fine-tuning enables LLMs to adapt more effectively to specialized tasks by retraining a pre-trained model on domain-specific datasets. Instead of relying solely on general-purpose knowledge, fine-tuning equips the model with the ability to recognize and reason about the unique characteristics of software security. This process is particularly important for three key reasons [155,156]. First, security vulnerabilities in source code typically manifest through recurring patterns, such as unsafe memory operations, unchecked user inputs, or flawed control flows. These subtle patterns must be explicitly learned by the model in order to distinguish vulnerable code from safe implementations. Second, programming languages differ significantly from natural language, as they follow strict syntactic and semantic rules. LLMs originally trained on mixed corpora often lack deep understanding of code structure; thus, fine-tuning helps them better parse program logic, dependency structures, and execution semantics. Third, vulnerability detection demands a high degree of precision, as even small misclassifications can lead to overlooked security flaws or excessive false alarms. Fine-tuning improves the model's ability to deliver reliable and accurate predictions in this context, making it a crucial step for applying LLMs to vulnerability detection.

Full Fine-Tuning (FFT). Full fine-tuning refers to the process of updating all model parameters during the training phase, which allows the LLMs to completely adapt to the downstream task. However, due to the high computational and memory costs of training large-scale models, most research has focused on relatively smaller architectures, typically below 15 billion parameters, such as CodeT5, CodeBERT, and UnixCoder. For example, Ding et al. [157] evaluated five models with fewer than 7B parameters and reported limited performance, achieving only a 0.21 F1-score on the PrimeVul dataset even when both training and validation were performed within the same distribution. Similarly, Guo et al. [158] fine-tuned CodeBERT for 50 epochs using FFT, which resulted in poor generalization across datasets: only 0.099 F1-score on PrimeVul, but a significantly higher 0.66 F1-score on the Choi2017 dataset. In contrast, Haurogne et al. [159] demonstrated improved performance with FFT, achieving a 0.69 F1-score on DiverseVul, while Purba et al. [160] reported an even stronger 0.73 F1-score for the task of buffer overflow detection. These findings suggest that while FFT allows full task adaptation, its effectiveness varies greatly depending on dataset diversity, model size, and task complexity, making it computationally expensive yet sometimes suboptimal compared to more efficient fine-tuning strategies.

Parameter-Efficient Fine-Tuning (PEFT). PEFT methods aim to adapt LLMs to downstream tasks by modifying only a small subset of parameters while keeping most of the pre-trained weights frozen. This significantly reduces the computational and memory costs while maintaining competitive performance. One common PEFT technique is the use of adapters, where lightweight trainable layers are inserted between the original transformer layers. For example, Yang et al. [161] applied adapters for fault localization and reported a 60% Top-5 accuracy, demonstrating their ability to guide models toward specialized tasks without full retraining. Another widely adopted technique is LoRA (Low-Rank Adapta-

tion), which represents weight updates as low-rank decompositions. LoRA has shown strong effectiveness across multiple vulnerability detection tasks: Du et al. [162] reported a 0.72 F1-score, while Guo et al. [158] achieved a remarkably high 0.97 F1-score on their respective datasets. An extension of this approach, QLoRA, combines LoRA with quantization techniques to further reduce memory requirements. Boi et al. [163] demonstrated that QLoRA can achieve 59% accuracy while maintaining low resource usage, highlighting its suitability for resource-constrained environments. Overall, PEFT methods strike a balance between efficiency and effectiveness, making them an increasingly attractive alternative to full fine-tuning, especially for large-scale LLMs applied to software vulnerability detection.

Generative Fine-Tuning. Generative fine-tuning adapts large language models for sequence-to-sequence tasks, where the goal is not just classification but the generation of structured outputs such as vulnerability explanations, patch suggestions, or the identification of vulnerable lines. Unlike discriminative approaches that only assign labels like vulnerable and non-vulnerable, generative fine-tuning allows models to produce richer outputs that enhance interpretability and support downstream tasks like automated repair. Yin et al. [164] demonstrated that fine-tuning pre-trained language models (LMs) for generative objectives often outperforms fine-tuning larger LLMs. For example, CodeT5+ achieved a ROUGE score of 0.722, substantially surpassing the performance of the much larger DeepSeek-Coder 6.7B, which reached only 0.425.

3.8. Evaluation Metrics

Classification Metrics. Evaluating vulnerability detection systems requires a range of metrics that capture both overall performance and specific trade-offs between false positives and false negatives [165,166]. Accuracy measures overall correctness and is defined as:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

While intuitive, accuracy can be misleading in highly imbalanced datasets, since a model predicting all samples as “non-vulnerable” may still achieve high accuracy. Precision, or the positive predictive value, is given by:

$$Precision = \frac{TP}{TP + FP} \quad (2)$$

Precision reflects the proportion of detected vulnerabilities that are truly vulnerable. High precision is essential when false alarms (false positives) are costly, such as in automated triage systems. Recall, or sensitivity, is defined as:

$$Recall = \frac{TP}{TP + FN} \quad (3)$$

It indicates how many of the actual vulnerabilities were correctly detected. High recall is critical in security applications because missing true vulnerabilities (false negatives) can lead to severe risks. F1-Score, the harmonic mean of precision and recall, balances these two metrics:

$$F1-Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (4)$$

Specificity (true negative rate) is the complement of recall and measures how effectively the model identifies safe code:

$$Specificity = \frac{TN}{TN + FP} \quad (5)$$

Generation and Explainability Metrics. For tasks that go beyond binary classification, such as generating vulnerability descriptions, explanations, or suggested fixes, evaluation requires different metrics. Bilingual Evaluation Understudy (BLEU) measures n-gram precision while applying a brevity penalty to prevent overly short outputs. It is widely used to assess generated vulnerability descriptions. Recall-Oriented Understudy for Gisting Evaluation (ROUGE) emphasizes recall by measuring overlap between generated and reference texts. Variants such as ROUGE-N, ROUGE-L, and ROUGE-S are used to evaluate explanation quality. METEOR goes beyond n-gram overlap by considering synonyms, stemming, and word order. Although less common in software engineering research, it can capture semantic similarity better than BLEU or ROUGE [167].

4. Insights, and Challenges

4.1. Limited Scope

Recent vulnerability detection research has been conducted on a variety of datasets that diverge in quantity, quality, and authenticity. Existing vulnerability datasets split into three categories: synthetic, semi-real, and real-world, each with a distinctive set of properties and applicability. Synthetic datasets are manually created and designed to represent specific vulnerability patterns, whereas real-world datasets are derived from software repositories, commits, and security patches that reflect appropriate software development configurations. The Juliet Test Suite is a synthetic benchmark that includes thousands of C/C++ and Java methods identified with CWE types. It provides clear annotations and extensive vulnerability coverage, making it excellent for early-stage model training and controlled benchmarking. However, Juliet's code fragments are artificially produced and context-independent, causing limited realism and poor applicability to real-world software. The Devign dataset, which has been generated from actual GitHub projects, comprises vulnerable-fixed code combinations that are useful for graph-based and patch-learning algorithms. Despite providing realistic code designs, it suffers from labeling noise, a small scale, and limited project variety. The dataset also contains duplicate samples. PrimeVul extends this approach by aggregating samples from several open-source repositories and providing function-level CVE- and CWE-mapped combinations, which facilitate multi-label categorization and cross-project assessment. Its breadth is constrained due to partial synthetic augmentation and inadequate commit information. Among large-scale real-world datasets, BigVul is one of the most comprehensive, comprising more than 350,000 C/C++ functions with real CVE-linked vulnerabilities mined from over 300 projects. It is widely used for LLMs pre-training and model generalization studies, though it faces class imbalance, noisy labeling, and heavy preprocessing requirements. The Reveal dataset focuses on Java projects, offering rich object-oriented code context and supporting language-specific and transfer-learning research, yet it has restricted language coverage and moderate sample size. Similarly, VUDENC combines vulnerable code with natural-language vulnerability descriptions, enabling reasoning and explanation generation for LLMs, but its limited data scale and language diversity constrain its general use. Draper VDISC contains real industrial C/C++ functions with fine-grained vulnerability metrics, suitable for interpretability and semantic-analysis research, although its medium scale and incomplete CWE labeling limit broader benchmarking. Although existing vulnerability datasets, whether synthetic, semi-real, or real-world have considerably improved research in automated vulnerability identification, the field's future growth is dependent on enhancing and increasing these corpora. Future research should focus on developing larger, cleaner, and more balanced multilingual datasets that reflect various project structures and complicated vulnerability meanings. Integrating commit-level context, fix history, and more comprehensive data will improve the realism and reasoning capacity of future datasets. Furthermore, standardizing

labeling techniques and creating consistent benchmarking frameworks would ensure consistency and comparability across studies. Combining synthetic creation with real-world mining might also assist with data shortage and imbalance concerns.

4.2. Generalization Across Languages and Domains

Most existing vulnerability detection models are trained and evaluated on a single programming language, yet real-world software systems are polyglot in nature. Models fine-tuned on one dataset often fail to generalize when applied to different languages or frameworks due to variations in syntax, semantics, and idiomatic practices. For example, vulnerabilities in memory-unsafe languages such as C/C++ differ significantly from those in managed languages like Python or Java. This challenge highlights the need for cross-language vulnerability datasets and multilingual fine-tuning strategies that enable LLMs to capture both language-specific patterns and universal vulnerability semantics.

4.3. Limited Vulnerability Type Coverage

Another important limitation in current LLM-based vulnerability detection research is the restricted coverage of vulnerability types. Most existing datasets and detection models focus primarily on a handful of common vulnerability classes such as buffer overflows, null pointer dereferences, or injection flaws. While these categories are important, real-world software systems face a much broader spectrum of security issues, including logic flaws, concurrency errors, race conditions, cryptographic misuses, insecure APIs, and supply-chain vulnerabilities. Many of these vulnerability classes are underrepresented or entirely absent in publicly available datasets due to the difficulty of labeling them and the lack of consistent ground truth. As a result, models trained on narrow datasets often demonstrate high performance for a limited set of CWE categories but fail to generalize to emerging or less frequent vulnerability types. This restricted scope creates a coverage gap between academic benchmarks and practical needs in industry, where vulnerabilities are diverse, context-dependent, and evolve rapidly with new technologies. Addressing this challenge requires the development of broader and more representative datasets that capture diverse vulnerability classes and encourage models to move beyond detecting only the most common patterns.

4.4. Limited Availability

A recurring challenge in LLM-based vulnerability detection research is the limited availability of datasets and trained models. Many studies rely on proprietary or internally curated datasets that are not released publicly due to confidentiality, licensing restrictions, or security concerns. This lack of open access hinders reproducibility and prevents other researchers from validating or extending prior work. Similarly, fine-tuned vulnerability detection models are often not made available, either because of intellectual property restrictions, the computational expense of hosting large checkpoints, or the sensitive nature of security-related models. As a result, researchers are frequently forced to reimplement baselines under different conditions, leading to inconsistencies in reported results. The unavailability of standardized datasets and pre-trained/fine-tuned models also slows progress in benchmarking, since it is difficult to make fair comparisons across methods. Addressing this challenge requires stronger community efforts toward open-source datasets, shared model repositories, and reproducible pipelines, similar to initiatives in NLP and computer vision. Such openness would accelerate innovation, foster collaboration, and improve the reliability of advances in vulnerability detection research.

4.5. Reproducibility and Benchmarking Gaps

A significant challenge in this research area is the lack of standardized, reproducible benchmarks. Many studies use different datasets, preprocessing pipelines, and evaluation metrics, making cross-comparison difficult. For example, some works report accuracy while others focus on F1-score, MCC, or ROC-AUC, often without clarifying dataset splits or deduplication strategies. This inconsistency leads to inflated claims and prevents fair evaluation of competing approaches. Establishing unified benchmarks and reproducible pipelines is therefore critical to advancing the field.

4.6. Lack of Interpretability

A major challenge in applying LLMs to vulnerability detection is the lack of interpretability and transparency in their predictions. While models such as CodeBERT, CodeT5, or LLaMA-based architectures can achieve high accuracy, they typically function as black boxes, providing little insight into why a particular code snippet is classified as vulnerable. For high-stakes domains like software security, this opacity is problematic: developers and auditors need to understand which lines of code are risky, which vulnerability category it corresponds to, and why the model reached its decision. Without interpretability, false positives can lead to wasted developer effort, while false negatives can leave critical vulnerabilities undetected. Although recent advances such as attention visualization, vulnerable line highlighting, and chain-of-thought prompting have attempted to improve explainability, these methods remain limited in consistency and reliability. In many cases, the explanations generated by LLMs may sound plausible but fail to align with the true vulnerability cause, creating a false sense of trust. This interpretability gap restricts the adoption of LLM-based vulnerability detection in industry, where accountability and explainability are critical for security audits, compliance, and developer trust. Bridging this gap requires research into more robust explainability techniques, such as neuro-symbolic reasoning, program slicing with interpretability, or integrating external knowledge into model outputs.

4.7. Real-World Performances

A persistent limitation of current LLM-based vulnerability detection systems is their performance gap between benchmark datasets and real-world software. Many models demonstrate strong results on curated datasets such as Juliet, Devign, or PrimeVul, where vulnerabilities are either synthetic or clearly labeled. However, when deployed on real-world projects—characterized by large, noisy, and heterogeneous codebases—the performance often degrades significantly. Real-world software frequently includes incomplete code fragments, project-specific libraries, macros, or mixed-language environments, which differ from the clean, self-contained samples used during training. Furthermore, vulnerabilities in production systems are often more subtle, context-dependent, and influenced by system-level interactions that are not captured in small benchmark functions. This results in models generating high false positive rates, missing rare but critical vulnerability types, or failing to generalize across projects and domains. The gap between laboratory benchmarks and industrial codebases limits the practical adoption of these systems in security auditing pipelines. Addressing this challenge requires not only larger and more representative datasets but also robust evaluation on real-world repositories, continual learning mechanisms to adapt to evolving code, and hybrid approaches that combine statistical learning with program analysis to handle complex contexts.

4.8. LLM Hallucination and Overfitting Issues

Another major challenge in applying LLMs for vulnerability detection is the tendency of models to hallucinate or overfit training data. Hallucination refers to the generation of confident but factually incorrect or irrelevant outputs, such as assigning a CWE label to code where no vulnerability exists, or producing vulnerability explanations that sound plausible but are technically inaccurate. In high-stakes security tasks, such hallucinations can be particularly dangerous, as they may mislead developers into believing that code is safe (false negatives) or raise unnecessary alarms (false positives). Overfitting, on the other hand, occurs when models memorize patterns from small or imbalanced datasets rather than learning generalizable vulnerability features. For example, models trained on benchmark datasets often latch onto surface-level token sequences rather than understanding deeper control- or data-flow dependencies, leading to degraded performance on unseen or real-world projects. These issues highlight a fundamental limitation of current LLMs: while they are powerful pattern recognizers, they lack grounding in program semantics and may generate outputs that do not align with actual execution behavior. Addressing hallucination and overfitting requires regularization techniques, adversarial evaluation, cross-dataset benchmarking, and hybrid approaches that combine LLMs with static or symbolic analysis to ensure that predictions remain both accurate and trustworthy.

4.9. Bias in LLM-Based Vulnerability Detection

Bias is an often-overlooked but significant challenge when applying LLMs to vulnerability detection. Since most LLMs are pre-trained on imbalanced and uncured corpora, they inherit biases present in the training data. For instance, publicly available vulnerability datasets frequently overrepresent certain vulnerability types while underrepresenting others such as cryptographic misuse, concurrency flaws, or logic errors. As a result, models may learn to prioritize detecting common vulnerability patterns while ignoring less frequent but equally critical classes, leading to type bias. Similarly, project and language bias arises because many datasets are dominated by a handful of open-source projects in C/C++, with limited representation from other languages or industrial codebases. This restricts generalization across diverse ecosystems. Furthermore, evaluation practices may unintentionally reinforce bias when models are repeatedly benchmarked on the same datasets, creating over-optimistic results that fail to reflect real-world conditions. Such biases not only undermine fairness and generalizability but also reduce trust in the model's outputs, as developers cannot be confident that all vulnerability categories are being treated equally. Addressing bias requires more balanced dataset construction, cross-language evaluations, fairness-aware training objectives, and transparent reporting of dataset limitations in future research.

4.10. GPU and Resource Limitations

Training and deploying LLMs for vulnerability detection is highly resource-intensive. State-of-the-art models often contain billions of parameters, requiring large amounts of GPU memory, high-bandwidth interconnects, and optimized distributed training frameworks. Full fine-tuning of models like CodeT5+, LLaMA-7B, or DeepSeek-Coder is often infeasible for many academic and industrial teams without access to specialized hardware. Even parameter-efficient methods such as LoRA or QLoRA reduce, but do not eliminate, the computational burden, as long training times, high memory usage, and significant energy costs remain. These GPU limitations restrict reproducibility, slow down experimentation cycles, and prevent widespread adoption of LLM-based vulnerability detectors. Moreover, inference at scale can also be prohibitively expensive if not carefully optimized. This

computational bottleneck makes it difficult to bridge the gap between research prototypes and deployable industry tools.

5. Future Directions

Despite recent advances in LLM-based vulnerability detection, numerous open issues remain unresolved. Future research should address these challenges by pursuing several complementary directions.

5.1. Data-Centric Development

Gap. Public corpora overrepresent a few CWE families and synthetic cases, limiting real-world generalization.

Direction. Future research should explore numerous, diversified sources to create organized, constantly revised datasets with rigorous deduplication, leakage control (project/time divides), and long-tail coverage (logic faults, crypto unlawful use, concurrency/races). Including flow facts (sources/sanitizers/sinks), span-level locations, and uncertainty ratings will result in more equitable training and evaluation. Benchmarks ought to include IID and OOD tracks with per-CWE metrics (macro-F1, AUROC, calibration) to ensure that models are awarded for breadth and robustness instead of overall reliability.

5.2. Neuro-Symbolic and Hybrid Approaches

Gap. LLMs capture patterns but lack semantic grounding (flows, control).

Direction. A potential strategy is to integrate code models with symbolic tools (AST/CFG/PDG, taint analysis, and logical parameters). Conventional pipelines include feature fusion (injecting flow/graph features), constraint-guided decoding (rejecting predictions that break flow rules), and post hoc verification (LLM + static checker). Evaluations should compare code-only, flow-only, and code + flow settings, assess evidence quality (sanitizer source to sink routes), and report hallucination reductions at a fixed retention rate.

5.3. Efficient and Resource-Aware Fine-Tuning

Gap. High VRAM cost blocks deployment in CI/IDEs.

Direction. Deployment in CI/IDEs demands low latency and small VRAM footprints. Parameter-efficient methods (LoRA/QLoRA/adapters), compression (quantization/pruning), and distillation should be standard, with cost reporting alongside accuracy (latency@batch, tokens/s, peak VRAM, energy per KLOC). Targets like ≤ 8 GB VRAM and sub-150 ms per function for IDEs make results actionable. The goal is Pareto-optimal detectors that preserve $\geq 95\%$ macro-F1 while doubling throughput.

5.4. Cross-Language and Multi-Modal Vulnerability Detection

Gap. Polyglot systems and mixed artifacts (code + commits + docs) are underused.

Direction. Modern codebases are multilingual and context-rich. Future models should learn universal vulnerability concepts and language-specific patterns in C/C++/Java/Python while including auxiliary modalities such as commits, issues/docs, IaC/configs, and runtime traces. Protocols should incorporate zero-shot cross-language testing, few-shot transfer, and multimodal capabilities to measure metadata benefits. Success is assessed by transfer $\Delta F1$ and improvements in fixable-bug recall after adding commits/traces.

5.5. Explainability and Trustworthy AI

Gap. Low adoption without actionable, auditable explanations.

Direction. Adoption relies on explanations that developers can check and act on. Detectors should produce highlighted spans, CWE maps, and evidence graphs to support

predictions, as well as calibrated confidence or conformal intervals. Human studies (time to triage, repair accuracy) and automated checks using static verifiers are required to validate utility. Faithfulness metrics (deletion/insertion tests) and calibrated uncertainty should be provided accurately, providing teams to trading precision for trust.

5.6. Fairness, Bias, and Robust Evaluation

Gap. Skewed labels and brittle models.

Direction. Addressing bias in datasets and models will be critical to ensure fair detection across vulnerability types, programming languages, and software domains. Standardized benchmarks, cross-dataset evaluations, and fairness-aware training objectives should become the norm. Additionally, rigorous adversarial evaluation strategies should be employed to assess robustness against code obfuscation, adversarial samples, and distributional shifts.

6. Conclusions

This survey provides a comprehensive and up-to-date overview of recent software vulnerability detection approaches, with an emphasis on Machine Learning (ML), Deep Learning (DL), and Large Language Models. While conventional rule-based and static techniques are still valuable for identifying known vulnerability patterns, their limited scalability and adaptability have prompted the change to data-driven solutions. ML and DL approaches automate feature extraction and representation learning, respectively, while LLMs enhance these capabilities by comprehending code semantics, reasoning over context, and creating patches and justifications. This review presents an additional clarification of the literature. First, it provides a uniform comparison framework that connects ML, DL, and LLM-based vulnerability detection algorithms using comparable evaluation standards. Second, it identifies previously neglected challenges such as data imbalance, interpretability, hallucination, and computing cost that limit the use of existing models. Third, it discusses upcoming topics including neuro-symbolic hybrid systems, parameter-efficient fine-tuning, continuous learning, and cross-language generalization. By articulating these insights, this review moves beyond descriptive summarization to identify concrete research opportunities and practical pathways for building next-generation vulnerability detection systems that are scalable, interpretable, and trustworthy for real-world software development.

Author Contributions: Conceptualization, M.S.A., M.S.H.S.; Data curation, Formal analysis, Investigation, M.S.H.S.; Methodology, M.S.H.S.; Project administration, M.S.A.; Resources, Software, M.S.A., M.S.H.S.; Supervision, M.S.A.; Validation, M.S.H.S.; Visualization, M.S.H.S.; Writing—original draft, M.S.H.S.; Writing—review editing, M.S.H.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Informed Consent Statement: Not applicable.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Wen, S.F.; Kowalski, S. A case study: Heartbleed vulnerability management and swedish municipalities. In Proceedings of the International Conference on Human Aspects of Information Security, Privacy, and Trust, Vancouver, BC, Canada, 9–14 July 2017; Springer International Publishing: Cham, Switzerland, 2017; pp. 414–431.
2. Shetty, R.; Choo, K.K.R.; Kaufman, R. Shellshock vulnerability exploitation and mitigation: A demonstration. In Proceedings of the International Conference on Applications and Techniques in Cyber Security and Intelligence, Ningbo, China, 31 January–31 December 2017; Springer International Publishing: Cham, Switzerland, 2017; pp. 338–350.

3. Chen, X.; Li, C.; Wang, D.; Wen, S.; Zhang, J.; Nepal, S.; Xiang, Y.; Ren, K. Android HIV: A study of repackaging malware for evading machine-learning detection. *IEEE Trans. Inf. Forensics Secur.* **2019**, *15*, 987–1001. [CrossRef]
4. Equifax Had Patch 2 Months Before Hack and Didn't Install It, Security Group Says. Available online: <https://www.contrastsecurity.com/security-influencers/still-making-headlines-struts-2-and-the-equifax-breach> (accessed on 5 January 2025).
5. Zhu, T.; Li, G.; Zhou, W.; Yu, P.S. Differentially private data publishing and analysis: A survey. *IEEE Trans. Knowl. Data Eng.* **2017**, *29*, 1619–1638. [CrossRef]
6. Zhu, T.; Xiong, P.; Li, G.; Zhou, W.; Yu, P.S. Differentially private model publishing in cyber physical systems. *Future Gener. Comput. Syst.* **2020**, *108*, 1297–1306. [CrossRef]
7. Lin, G.; Wen, S.; Han, Q.L.; Zhang, J.; Xiang, Y. Software vulnerability detection using deep neural networks: A survey. *Proc. IEEE* **2020**, *108*, 1825–1848. [CrossRef]
8. Engler, D.; Chen, D.Y.; Hallem, S.; Chou, A.; Chelf, B. Bugs as deviant behavior: A general approach to inferring errors in systems code. *ACM SIGOPS Oper. Syst. Rev.* **2001**, *35*, 57–72. [CrossRef]
9. Wheeler, D.A. Flawfinder. 2016. Available online: <https://dwheeler.com/flawfinder/> (accessed on 5 January 2025).
10. Hochreiter, S.; Schmidhuber, J. Long short-term memory. *Neural Comput.* **1997**, *9*, 1735–1780. [CrossRef]
11. Kim, S.; Woo, S.; Lee, H.; Oh, H. Vuddy: A scalable approach for vulnerable code clone discovery. In Proceedings of the 2017 IEEE Symposium on Security and Privacy (SP), San Jose, CA, USA, 22–26 May 2017; IEEE: New York, NY, USA, 2017; pp. 595–614.
12. Cadar, C.; Dunbar, D.; Engler, D.R. Klee: Unassisted and automatic generation of high-coverage tests for complex systems programs. In Proceedings of the OSDI, San Diego, CA, USA, 8–10 December 2008; Volume 8, pp. 209–224.
13. Ramos, D.A.; Engler, D. {Under-Constrained} symbolic execution: Correctness checking for real code. In Proceedings of the 24th USENIX Security Symposium (USENIX Security 15), Austin, TX, USA, 10–12 August 2015; pp. 49–64.
14. Thanassis, H.A.; Kil, C.S.; David, B. Aeg: Automatic exploit generation. In Proceedings of the Network and Distributed System Security Symposium, San Diego, CA, USA, 6–9 February 2011.
15. Li, Z.; Zou, D.; Xu, S.; Ou, X.; Jin, H.; Wang, S.; Deng, Z.; Zhong, Y. Vuldeepecker: A deep learning-based system for vulnerability detection. *arXiv* **2018**, arXiv:1801.01681.
16. Sabottke, C.; Suci, O.; Dumitras, T. Vulnerability disclosure in the age of social media: Exploiting twitter for predicting {Real-World} exploits. In Proceedings of the 24th USENIX Security Symposium (USENIX Security 15), Austin, TX, USA, 10–12 August 2015; pp. 1041–1056.
17. Sutton, M.; Greene, A.; Amini, P. *Fuzzing: Brute Force Vulnerability Discovery*; Pearson Education: London, UK, 2007.
18. Newsome, J.; Song, D.X. Dynamic taint analysis for automatic detection, analysis, and signature generation of exploits on commodity software. In Proceedings of the Network and Distributed System Security (NDSS) Symposium, San Diego, CA, USA, 3–4 February 2005; Volume 5, pp. 3–4.
19. Portokalidis, G.; Slowinska, A.; Bos, H. Argos: An emulator for fingerprinting zero-day attacks for advertised honeypots with automatic signature generation. *ACM SIGOPS Oper. Syst. Rev.* **2006**, *40*, 15–27. [CrossRef]
20. Coulter, R.; Han, Q.L.; Pan, L.; Zhang, J.; Xiang, Y. Data-driven cyber security in perspective—Intelligent traffic analysis. *IEEE Trans. Cybern.* **2019**, *50*, 3081–3093. [CrossRef]
21. Ghaffarian, S.M.; Shahriari, H.R. Software vulnerability analysis and discovery using machine-learning and data-mining techniques: A survey. *ACM Comput. Surv.* **2017**, *50*, 1–36. [CrossRef]
22. Liu, L.; De Vel, O.; Han, Q.L.; Zhang, J.; Xiang, Y. Detecting and preventing cyber insider threats: A survey. *IEEE Commun. Surv. Tutor.* **2018**, *20*, 1397–1417. [CrossRef]
23. Sun, N.; Zhang, J.; Rimba, P.; Gao, S.; Zhang, L.Y.; Xiang, Y. Data-driven cybersecurity incident prediction: A survey. *IEEE Commun. Surv. Tutor.* **2018**, *21*, 1744–1772. [CrossRef]
24. Hossain, S.; Zulkernine, M. Mitigating program security vulnerabilities: Approaches and challenges. *ACM Comput. Surv.* **2012**, *44*, 1–46.
25. Malhotra, R. A systematic review of machine learning techniques for software fault prediction. *Appl. Soft Comput.* **2015**, *27*, 504–518. [CrossRef]
26. Liu, B.; Shi, L.; Cai, Z.; Li, M. Software vulnerability discovery techniques: A survey. In Proceedings of the 2012 Fourth International Conference on Multimedia Information Networking and Security, Nanjing, China, 2–4 November 2012; IEEE: New York, NY, USA, 2012; pp. 152–156.
27. Jie, G.; Xiao-Hui, K.; Qiang, L. Survey on software vulnerability analysis method based on machine learning. In Proceedings of the 2016 IEEE First International Conference on Data Science in Cyberspace (DSC), Changsha, China, 13–16 June 2016; IEEE: New York, NY, USA, 2016; pp. 642–647.
28. Allamanis, M.; Barr, E.T.; Devanbu, P.; Sutton, C. A survey of machine learning for big code and naturalness. *ACM Comput. Surv.* **2018**, *51*, 1–37. [CrossRef]
29. Shimmi, S.; Okhravi, H.; Rahimi, M. AI-Based Software Vulnerability Detection: A Systematic Literature Review. *arXiv* **2025**, arXiv:2506.10280. [CrossRef]

30. Hanif, H.; Nasir, M.H.N.M.; Ab Razak, M.F.; Firdaus, A.; Anuar, N.B. The rise of software vulnerability: Taxonomy of software vulnerabilities detection and machine learning approaches. *J. Netw. Comput. Appl.* **2021**, *179*, 103009. [\[CrossRef\]](#)
31. Shiri Harzevili, N.; Boaye Belle, A.; Wang, J.; Wang, S.; Jiang, Z.M.; Nagappan, N. A systematic literature review on automated software vulnerability detection using machine learning. *ACM Comput. Surv.* **2024**, *57*, 1–36. [\[CrossRef\]](#)
32. Zeng, P.; Lin, G.; Pan, L.; Tai, Y.; Zhang, J. Software vulnerability analysis and discovery using deep learning techniques: A survey. *IEEE Access* **2020**, *8*, 197158–197172. [\[CrossRef\]](#)
33. Lomio, F.; Iannone, E.; De Lucia, A.; Palomba, F.; Lenarduzzi, V. Just-in-time software vulnerability detection: Are we there yet? *J. Syst. Softw.* **2022**, *188*, 111283. [\[CrossRef\]](#)
34. Liu, M.; Zhang, B.; Chen, W.; Zhang, X. A survey of exploitation and detection methods of XSS vulnerabilities. *IEEE Access* **2019**, *7*, 182004–182016. [\[CrossRef\]](#)
35. Chakraborty, S.; Krishna, R.; Ding, Y.; Ray, B. Deep learning based vulnerability detection: Are we there yet? *IEEE Trans. Softw. Eng.* **2021**, *48*, 3280–3296. [\[CrossRef\]](#)
36. Zheng, W.; Semasaba, A.O.A.; Wu, X.; Agyemang, S.A.; Liu, T.; Ge, Y. Representation vs. Model: What matters most for source code vulnerability detection. In Proceedings of the 2021 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER), Honolulu, HI, USA, 9–12 March 2021; IEEE: New York, NY, USA, 2021; pp. 647–653.
37. Zhao, J.; Zhu, K.; Yu, L.; Huang, H.; Lu, Y. Yama: Precise Opcode-Based Data Flow Analysis for Detecting PHP Applications Vulnerabilities. *IEEE Trans. Inf. Forensics Secur.* **2025**, *20*, 7748–7763. [\[CrossRef\]](#)
38. Ji, Y.; Dai, T.; Zhou, Z.; Tang, Y.; He, J. Artemis: Toward Accurate Detection of Server-Side Request Forgeries through LLM-Assisted Inter-Procedural Path-Sensitive Taint Analysis. *Proc. ACM Program. Lang.* **2025**, *9*, 1349–1377. [\[CrossRef\]](#)
39. Real World Example. Available online: <https://www.invokesec.com/2025/01/13/a-real-world-example-of-blind-sqli/> (accessed on 5 April 2025).
40. SQL Injection. Available online: <https://www.radware.com/cyberpedia/application-security/sql-injection/> (accessed on 5 April 2025).
41. Vulnerability Reports. Available online: <https://www.edgescan.com/wp-content/uploads/2024/03/2023-Vulnerability-Statistics-Report.pdf> (accessed on 5 April 2025).
42. Vulnerable XSS. Available online: <https://www.acunetix.com/blog/articles/33-websites-webapps-vulnerable-xss/> (accessed on 5 April 2025).
43. Buffer Overflow. Available online: <https://www.invicti.com/blog/web-security/2024-cwe-top-25-list-xss-sqli-buffer-overflows/> (accessed on 5 April 2025).
44. Security Patches. Available online: <https://www.wired.com/story/apple-google-moveit-security-patches-june-2023-critical-update/> (accessed on 5 April 2025).
45. Top 10 OWASP. Available online: <https://owasp.org/Top10/> (accessed on 5 April 2025).
46. IEEE. *IEEE Standard Glossary of Software Engineering Terminology*; IEEE: New York, NY, USA, 1990.
47. Chess, B.; McGraw, G. Static analysis for security. *IEEE Secur. Priv.* **2004**, *2*, 76–79. [\[CrossRef\]](#)
48. Ernst, M.D. Static and dynamic analysis: Synergy and duality. In Proceedings of the WODA 2003: ICSE Workshop on Dynamic Analysis, Portland, OR, USA, 3–10 May 2003; pp. 24–27.
49. Ayewah, N.; Pugh, W.; Hovemeyer, D.; Morgenthaler, J.D.; Penix, J. Using static analysis to find bugs. *IEEE Softw.* **2008**, *25*, 22–29. [\[CrossRef\]](#)
50. Harzevili, N.S.; Shin, J.; Wang, J.; Wang, S.; Nagappan, N. Automatic static vulnerability detection for machine learning libraries: Are we there yet? In Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering (ISSRE), Florence, Italy, 9–12 October 2023; IEEE: New York, NY, USA, 2023; pp. 795–806.
51. Akram, J.; Luo, P. SQVDT: A scalable quantitative vulnerability detection technique for source code security assessment. *Softw. Pract. Exp.* **2021**, *51*, 294–318. [\[CrossRef\]](#)
52. Bowman, B.; Huang, H.H. VGRAPH: A robust vulnerable code clone detection system using code property triplets. In Proceedings of the 2020 IEEE European Symposium on Security and Privacy (EuroS&P), Genoa, Italy, 7–11 September 2020; IEEE: New York, NY, USA, 2020; pp. 53–69.
53. Rahaman, S.; Xiao, Y.; Afrose, S.; Shaon, F.; Tian, K.; Frantz, M.; Kantarcioglu, M.; Yao, D. Cryptoguard: High precision detection of cryptographic vulnerabilities in massive-sized java projects. In Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, London, UK, 11–15 November 2019; pp. 2455–2472.
54. Miller, B.P.; Fredriksen, L.; So, B. An empirical study of the reliability of UNIX utilities. *Commun. ACM* **1990**, *33*, 32–44. [\[CrossRef\]](#)
55. Oehlert, P. Violating Assumptions with Fuzzing. *IEEE Secur. Priv.* **2005**, *3*, 58–62. [\[CrossRef\]](#)
56. Lanzi, A.; Martignoni, L.; Monga, M.; Paleari, R. A smart fuzzer for x86 executables. In Proceedings of the Third International Workshop on Software Engineering for Secure Systems (SESS’07: ICSE Workshops 2007), Minneapolis, MN, USA, 20–26 May 2007; IEEE: New York, NY, USA, 2007; p. 7.

57. Kang, W.; Son, B.; Heo, K. Tracer: Signature-based static analysis for detecting recurring vulnerabilities. In Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, Los Angeles, CA, USA, 7–11 November 2022; pp. 1695–1708.
58. Zagane, M.; Abdi, M.K.; Alenezi, M. A new approach to locate software vulnerabilities using code metrics. *Int. J. Softw. Innov. (IJSI)* **2020**, *8*, 82–95. [[CrossRef](#)]
59. Salimi, S.; Ebrahimzadeh, M.; Kharrazi, M. Improving real-world vulnerability characterization with vulnerable slices. In Proceedings of the 16th ACM International Conference on Predictive Models and Data Analytics in Software Engineering, Virtual, 8–9 November 2020; pp. 11–20.
60. Kosker, Y.; Turhan, B.; Bener, A. An expert system for determining candidate software classes for refactoring. *Expert Syst. Appl.* **2009**, *36*, 10000–10003. [[CrossRef](#)]
61. Kumar, L.; Sureka, A. Application of LSSVM and SMOTE on seven open source projects for predicting refactoring at class level. In Proceedings of the 2017 24th Asia-Pacific Software Engineering Conference (APSEC), Nanjing, China, 4–8 December 2017; IEEE: New York, NY, USA, 2017; pp. 90–99.
62. Nyamawe, A.S.; Liu, H.; Niu, N.; Umer, Q.; Niu, Z. Automated recommendation of software refactorings based on feature requests. In Proceedings of the 2019 IEEE 27th International Requirements Engineering Conference (RE), Jeju, Republic of Korea, 23–27 September 2019; IEEE: New York, NY, USA, 2019; pp. 187–198.
63. Cao, D.; Huang, J.; Zhang, X.; Liu, X. FTCLNet: Convolutional LSTM with Fourier transform for vulnerability detection. In Proceedings of the 2020 IEEE 19th International Conference on Trust, Security and Privacy in Computing and Communications (TrustCom), Guangzhou, China, 29 December 2020–1 January 2021; IEEE: New York, NY, USA, 2020; pp. 539–546.
64. Dam, H.K.; Tran, T.; Pham, T.; Ng, S.W.; Grundy, J.; Ghose, A. Automatic feature learning for predicting vulnerable software components. *IEEE Trans. Softw. Eng.* **2018**, *47*, 67–85. [[CrossRef](#)]
65. Jeon, S.; Kim, H.K. AutoVAS: An automated vulnerability analysis system with a deep learning approach. *Comput. Secur.* **2021**, *106*, 102308. [[CrossRef](#)]
66. Saccente, N.; Dehlinger, J.; Deng, L.; Chakraborty, S.; Xiong, Y. Project achilles: A prototype tool for static method-level vulnerability detection of java source code using a recurrent neural network. In Proceedings of the 2019 34th IEEE/ACM International Conference on Automated Software Engineering Workshop (ASEW), San Diego, CA, USA, 11–15 November 2019; IEEE: New York, NY, USA, 2019; pp. 114–121.
67. Xiaomeng, W.; Tao, Z.; Runpu, W.; Wei, X.; Changyu, H. CPGVA: Code property graph based vulnerability analysis by deep learning. In Proceedings of the 2018 10th International Conference on Advanced Infocomm Technology (ICAIT), Stockholm, Sweden, 12–15 August 2018; IEEE: New York, NY, USA, 2018; pp. 184–188.
68. Ziems, N.; Wu, S. Security vulnerability detection using deep learning natural language processing. In Proceedings of the IEEE INFOCOM 2021-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Vancouver, BC, Canada, 10–13 May 2021; IEEE: New York, NY, USA, 2021; pp. 1–6.
69. Chang, J.; Ma, Z.; Cao, B.; Zhu, E. VDDA: An effective software vulnerability detection model based on deep learning and attention mechanism. In Proceedings of the 2023 26th International Conference on Computer Supported Cooperative Work in Design (CSCWD), Rio de Janeiro, Brazil, 24–26 May 2023; IEEE: New York, NY, USA, 2023; pp. 474–479.
70. Chen, Y.; Liu, Z. Hlt: A hierarchical vulnerability detection model based on transformer. In Proceedings of the 2022 4th International Conference on Data Intelligence and Security (ICDIS), Shenzhen, China, 24–26 August 2022; IEEE: New York, NY, USA, 2022; pp. 50–54.
71. Du, G.; Chen, L.; Wu, T.; Zheng, X.; Cui, N.; Shi, G. Cross domain on snippets: BiLSTM-TextCNN based vulnerability detection with domain adaptation. In Proceedings of the 2023 26th International Conference on Computer Supported Cooperative Work in Design (CSCWD), Rio de Janeiro, Brazil, 24–26 May 2023; IEEE: New York, NY, USA, 2023; pp. 1896–1901.
72. Guo, W.; Fang, Y.; Huang, C.; Ou, H.; Lin, C.; Guo, Y. HyVulDect: A hybrid semantic vulnerability mining system based on graph neural network. *Comput. Secur.* **2022**, *121*, 102823. [[CrossRef](#)]
73. Li, Z.; Zou, D.; Xu, S.; Jin, H.; Zhu, Y.; Chen, Z. Sysevr: A framework for using deep learning to detect software vulnerabilities. *IEEE Trans. Dependable Secur. Comput.* **2021**, *19*, 2244–2258. [[CrossRef](#)]
74. Li, Z.; Zou, D.; Xu, S.; Chen, Z.; Zhu, Y.; Jin, H. Vuldeelocator: A deep learning-based fine-grained vulnerability detector. *IEEE Trans. Dependable Secur. Comput.* **2021**, *19*, 2821–2837. [[CrossRef](#)]
75. Feng, H.; Fu, X.; Sun, H.; Wang, H.; Zhang, Y. Efficient vulnerability detection based on abstract syntax tree and deep learning. In Proceedings of the IEEE INFOCOM 2020-IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS), Toronto, ON, Canada, 6–9 July 2020; IEEE: New York, NY, USA, 2020; pp. 722–727.
76. Cao, S.; Sun, X.; Bo, L.; Wu, R.; Li, B.; Tao, C. MVD: Memory-related vulnerability detection based on flow-sensitive graph neural networks. In Proceedings of the 44th International Conference on Software Engineering, Pittsburgh, PA, USA, 21–29 May 2022; pp. 1456–1468.

77. De Kraker, W.; Vranken, H.; Hommersom, A. GLICE: Combining graph neural networks and program slicing to improve software vulnerability detection. In Proceedings of the 2023 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW), Delft, The Netherlands, 3–7 July 2023; IEEE: New York, NY, USA, 2023; pp. 34–41.
78. Duan, X.; Wu, J.; Du, M.; Luo, T.; Yang, M.; Wu, Y. MultiCode: A Unified Code Analysis Framework based on Multi-type and Multi-granularity Semantic Learning. In Proceedings of the 2021 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Wuhan, China, 25–28 October 2021; IEEE: New York, NY, USA, 2021; pp. 359–364.
79. Hin, D.; Kan, A.; Chen, H.; Babar, M.A. Linevd: Statement-level vulnerability detection using graph neural networks. In Proceedings of the 19th International Conference on Mining Software Repositories, Pittsburgh, PA, USA, 23–24 May 2022; pp. 596–607.
80. Li, M.; Li, C.; Li, S.; Wu, Y.; Zhang, B.; Wen, Y. ACGVD: Vulnerability detection based on comprehensive graph via graph neural network with attention. In Proceedings of the International Conference on Information and Communications Security, Chongqing, China, 19–21 November 2021; Springer International Publishing: Cham, Switzerland, 2021; pp. 243–259.
81. Luo, Y.; Xu, W.; Xu, D. Compact abstract graphs for detecting code vulnerability with GNN models. In Proceedings of the 38th Annual Computer Security Applications Conference, Austin, TX, USA, 5–9 December 2022; pp. 497–507.
82. Nguyen, V.A.; Nguyen, D.Q.; Nguyen, V.; Le, T.; Tran, Q.H.; Phung, D. Regvd: Revisiting graph neural networks for vulnerability detection. In Proceedings of the ACM/IEEE 44th International Conference on Software Engineering: Companion Proceedings, Pittsburgh, PA, USA, 21–29 May 2022; pp. 178–182.
83. Song, Z.; Wang, J.; Liu, S.; Fang, Z.; Yang, K. HGVul: A Code Vulnerability Detection Method Based on Heterogeneous Source-Level Intermediate Representation. *Secur. Commun. Netw.* **2022**, *2022*, 1919907. [\[CrossRef\]](#)
84. Şahin, C.B. Semantic-based vulnerability detection by functional connectivity of gated graph sequence neural networks. *Soft Comput.* **2023**, *27*, 5703–5719. [\[CrossRef\]](#)
85. Wang, H.; Ye, G.; Tang, Z.; Tan, S.H.; Huang, S.; Fang, D.; Feng, Y.; Bian, L.; Wang, Z. Combining graph-based learning with automated data collection for code vulnerability detection. *IEEE Trans. Inf. Forensics Secur.* **2020**, *16*, 1943–1958. [\[CrossRef\]](#)
86. Wu, B.; Liu, S.; Xiao, Y.; Li, Z.; Sun, J.; Lin, S.W. Learning program semantics for vulnerability detection via vulnerability-specific inter-procedural slicing. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco, CA, USA, 3–9 December 2023; pp. 1371–1383.
87. Wu, T.; Chen, L.; Du, G.; Zhu, C.; Cui, N.; Shi, G. Inductive vulnerability detection via gated graph neural network. In Proceedings of the 2022 IEEE 25th International Conference on Computer Supported Cooperative Work in Design (CSCWD), Hangzhou, China, 4–6 May 2022; IEEE: New York, NY, USA, 2022; pp. 519–524.
88. Yang, J.; Ruan, O.; Zhang, J. Tensor-based gated graph neural network for automatic vulnerability detection in source code. *Softw. Test. Verif. Reliab.* **2024**, *34*, e1867. [\[CrossRef\]](#)
89. Zheng, W.; Jiang, Y.; Su, X. Vu1SPG: Vulnerability detection based on slice property graph representation learning. In Proceedings of the 2021 IEEE 32nd International Symposium on Software Reliability Engineering (ISSRE), Wuhan, China, 25–28 October 2021; IEEE: New York, NY, USA, 2021; pp. 457–467.
90. Dong, Y.; Tang, Y.; Cheng, X.; Yang, Y.; Wang, S. SedSVD: Statement-level software vulnerability detection based on Relational Graph Convolutional Network with subgraph embedding. *Inf. Softw. Technol.* **2023**, *158*, 107168. [\[CrossRef\]](#)
91. Cheng, X.; Wang, H.; Hua, J.; Zhang, M.; Xu, G.; Yi, L.; Sui, Y. Static detection of control-flow-related vulnerabilities using graph embedding. In Proceedings of the 2019 24th International Conference on Engineering of Complex Computer Systems (ICECCS), Guangzhou, China, 10–13 November 2019; IEEE: New York, NY, USA, 2019; pp. 41–50.
92. Wu, J. *Introduction to Convolutional Neural Networks*; National Key Lab for Novel Software Technology, Nanjing University: Nanjing, China, 2017; Volume 5, p. 495.
93. Bilgin, Z.; Ersoy, M.A.; Soykan, E.U.; Tomur, E.; Çomak, P.; Karaçay, L. Vulnerability prediction from source code using machine learning. *IEEE Access* **2020**, *8*, 150672–150684. [\[CrossRef\]](#)
94. Li, X.; Wang, L.; Xin, Y.; Yang, Y.; Chen, Y. Automated vulnerability detection in source code using minimum intermediate representation learning. *Appl. Sci.* **2020**, *10*, 1692. [\[CrossRef\]](#)
95. Liu, Y.; Wang, Y. An Effective Software Vulnerability Detection Method Based on Devised Deep-Learning Model to Fix the Vague Separation. In Proceedings of the 2022 3rd International Symposium on Big Data and Artificial Intelligence, Singapore, 9–10 December 2022; pp. 90–95.
96. Mim, R.S.; Khatun, A.; Ahammed, T.; Sakib, K. Impact of Centrality on Automated Vulnerability Detection Using Convolutional Neural Network. In Proceedings of the 2023 International Conference on Information and Communication Technology for Sustainable Development (ICICT4SD), Dhaka, Bangladesh, 21–23 September 2023; IEEE: New York, NY, USA, 2023; pp. 331–335.
97. Peng, B.; Liu, Z.; Zhang, J.; Su, P. CEVulDet: A code edge representation learnable vulnerability detector. In Proceedings of the 2023 International Joint Conference on Neural Networks (IJCNN), Gold Coast, Australia, 18–23 June 2023; IEEE: New York, NY, USA, 2023; pp. 1–8.

98. Russell, R.; Kim, L.; Hamilton, L.; Lazovich, T.; Harer, J.; Ozdemir, O.; Ellingwood, P.; McConley, M. Automated vulnerability detection in source code using deep representation learning. In Proceedings of the 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA), Orlando, FL, USA, 17–20 December 2018; IEEE: New York, NY, USA, 2018; pp. 757–762.
99. Tang, Z.; Hu, Q.; Hu, Y.; Kuang, W.; Chen, J. SEVulDet: A semantics-enhanced learnable vulnerability detector. In Proceedings of the 2022 52nd Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), Baltimore, MD, USA, 27–30 June 2022; IEEE: New York, NY, USA, 2022; pp. 150–162.
100. Xuan, C.D. A new approach to software vulnerability detection based on CPG analysis. *Cogent Eng.* **2023**, *10*, 2221962. [\[CrossRef\]](#)
101. Zhao, W.X.; Zhou, K.; Li, J.; Tang, T.; Wang, X.; Hou, Y.; Min, Y.; Zhang, B.; Zhang, J.; Dong, Z.; et al. A survey of large language models. *arXiv* **2023**, arXiv:2303.18223.
102. Ashish, V. Attention is all you need. *Adv. Neural Inf. Process. Syst.* **2017**, *30*, 1.
103. Bommasani, R. On the opportunities and risks of foundation models. *arXiv* **2021**, arXiv:2108.07258. [\[CrossRef\]](#)
104. Kaplan, J.; McCandlish, S.; Henighan, T.; Brown, T.B.; Chess, B.; Child, R.; Gray, S.; Radford, A.; Wu, J.; Amodei, D. Scaling laws for neural language models. *arXiv* **2020**, arXiv:2001.08361. [\[CrossRef\]](#)
105. Dai, H.; Liu, Z.; Liao, W.; Huang, X.; Cao, Y.; Wu, Z.; Zhao, L.; Xu, S.; Zeng, F.; Liu, W.; et al. Auggpt: Leveraging chatgpt for text data augmentation. *IEEE Trans. Big Data* **2025**, *11*, 907–918. [\[CrossRef\]](#)
106. Pan, S.; Luo, L.; Wang, Y.; Chen, C.; Wang, J.; Wu, X. Unifying large language models and knowledge graphs: A roadmap. *IEEE Trans. Knowl. Data Eng.* **2024**, *36*, 3580–3599. [\[CrossRef\]](#)
107. Devlin, J.; Chang, M.W.; Lee, K.; Toutanova, K. Bert: Pre-training of deep bidirectional transformers for language understanding. In Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, 2–7 June 2019; Volume 1 (long and short papers), pp. 4171–4186.
108. Feng, Z.; Guo, D.; Tang, D.; Duan, N.; Feng, X.; Gong, M.; Shou, L.; Qin, B.; Liu, T.; Jiang, D.; et al. Codebert: A pre-trained model for programming and natural languages. *arXiv* **2020**, arXiv:2002.08155.
109. Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S.; et al. Graphcodebert: Pre-training code representations with data flow. *arXiv* **2020**, arXiv:2009.08366.
110. Kanade, A.; Maniatis, P.; Balakrishnan, G.; Shi, K. Learning and evaluating contextual embedding of source code. In Proceedings of the International Conference on Machine Learning, PMLR, Virtual, 13–18 July 2020; pp. 5110–5121.
111. Hanif, H.; Maffei, S. VulBERTa: Simplified source code pre-training for vulnerability detection. In Proceedings of the 2022 International Joint Conference on Neural Networks (IJCNN), Padua, Italy, 18–23 July 2022; IEEE: New York, NY, USA, 2022; pp. 1–8.
112. Zhou, X.; Xu, B.; Han, D.; Yang, Z.; He, J.; Lo, D. CCBERT: Self-supervised code change representation learning. In Proceedings of the 2023 IEEE International Conference on Software Maintenance and Evolution (ICSME), Bogotá, Colombia, 1–6 October 2023; IEEE: New York, NY, USA, 2023; pp. 182–193.
113. Ding, Y.; Wu, Q.; Li, Y.; Wang, D.; Huang, J. Leveraging Deep Learning Models for Cross-function Null Pointer Risks Detection. In Proceedings of the 2023 IEEE International Conference on Artificial Intelligence Testing (AITest), Athens, Greece, 17–20 July 2023; IEEE: New York, NY, USA, 2023; pp. 107–113.
114. Kim, S.; Choi, J.; Ahmed, M.E.; Nepal, S.; Kim, H. VulDeBERT: A vulnerability detection system using bert. In Proceedings of the 2022 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW), Charlotte, NC, USA, 31 October–3 November 2022; IEEE: New York, NY, USA, 2022; pp. 69–74.
115. Wu, T.; Chen, L.; Du, G.; Zhu, C.; Cui, N.; Shi, G. CDNLM: Clustering-based data normalization method for automated vulnerability detection. *Comput. J.* **2024**, *67*, 1538–1549. [\[CrossRef\]](#)
116. Quan, V.L.A.; Phat, C.T.; Van Nguyen, K.; The Duy, P.; Pham, V.H. Xgy-bert: Leveraging contextualized language model and graph neural network for efficient software vulnerability detection. *J. Supercomput.* **2025**, *81*, 750. [\[CrossRef\]](#)
117. Ahmad, W.U.; Chakraborty, S.; Ray, B.; Chang, K.W. Unified pre-training for program understanding and generation. *arXiv* **2021**, arXiv:2103.06333. [\[CrossRef\]](#)
118. Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P.J. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.* **2020**, *21*, 1–67.
119. Wang, Y.; Wang, W.; Joty, S.; Hoi, S.C. Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation. *arXiv* **2021**, arXiv:2109.00859.
120. Guo, D.; Lu, S.; Duan, N.; Wang, Y.; Zhou, M.; Yin, J. Unixcoder: Unified cross-modal pre-training for code representation. *arXiv* **2022**, arXiv:2203.03850.
121. Chakraborty, S.; Ahmed, T.; Ding, Y.; Devanbu, P.T.; Ray, B. NatGen: Generative pre-training by “naturalizing” source code. In Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Singapore, 14–18 November 2022; pp. 18–30.

122. Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I. Language models are unsupervised multitask learners. *OpenAI Blog* **2019**, *1*, 9.
123. Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; Agarwal, S. Language models are few-shot learners. *arXiv* **2020**, arXiv:2005.14165. [[CrossRef](#)]
124. OpenAI 2022. GPT-3.5. Available online: <https://platform.openai.com/docs/models> (accessed on 14 May 2025).
125. Achiam, J.; Adler, S.; Agarwal, S.; Ahmad, L.; Akkaya, I.; Aleman, F.L.; Almeida, D.; Altenschmidt, J.; Altman, S.; Anadkat, S.; et al. Gpt-4 technical report. *arXiv* **2023**, arXiv:2303.08774. [[CrossRef](#)]
126. Lu, S.; Guo, D.; Ren, S.; Huang, J.; Svyatkovskiy, A.; Blanco, A.; Clement, C.; Drain, D.; Jiang, D.; Tang, D.; et al. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv* **2021**, arXiv:2102.04664. [[CrossRef](#)]
127. Chen, M.; Tworek, J.; Jun, H.; Yuan, Q.; Pinto, H.P.D.O.; Kaplan, J.; Edwards, H.; Burda, Y.; Joseph, N.; Brockman, G.; et al. Evaluating large language models trained on code. *arXiv* **2021**, arXiv:2107.03374. [[CrossRef](#)]
128. Xu, F.F.; Alon, U.; Neubig, G.; Hellendoorn, V.J. A systematic evaluation of large language models of code. In Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming, San Diego, CA, USA, 13 June 2022; pp. 1–10.
129. Fried, D.; Aghajanyan, A.; Lin, J.; Wang, S.; Wallace, E.; Shi, F.; Zhong, R.; Yih, W.T.; Zettlemoyer, L.; Lewis, M. Incoder: A generative model for code infilling and synthesis. *arXiv* **2022**, arXiv:2204.05999.
130. Nijkamp, E.; Pang, B.; Hayashi, H.; Tu, L.; Wang, H.; Zhou, Y.; Savarese, S.; Xiong, C. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv* **2022**, arXiv:2203.13474.
131. Roziere, B.; Gehring, J.; Gloeckle, F.; Sootla, S.; Gat, I.; Tan, X.E.; Adi, Y.; Liu, J.; Sauvestre, R.; Remez, T.; et al. Code llama: Open foundation models for code. *arXiv* **2023**, arXiv:2308.12950.
132. Li, R.; Allal, L.B.; Zi, Y.; Muennighoff, N.; Kocetkov, D.; Mou, C.; Marone, M.; Akiki, C.; Li, J.; Chim, J.; et al. Starcoder: May the source be with you! *arXiv* **2023**, arXiv:2305.06161. [[CrossRef](#)]
133. Zhou, X.; Zhang, T.; Lo, D. Large language model for vulnerability detection: Emerging results and future directions. In Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results, Lisbon, Portugal, 14–20 April 2024; pp. 47–51.
134. Cao, S.; Sun, X.; Bo, L.; Wei, Y.; Li, B. Bgnn4vd: Constructing bidirectional graph neural-network for vulnerability detection. *Inf. Softw. Technol.* **2021**, *136*, 106576. [[CrossRef](#)]
135. Wang, S.; Liu, T.; Tan, L. Automatically learning semantic features for defect prediction. In Proceedings of the 38th International Conference on Software Engineering, Austin, TX, USA, 14–22 May 2016; pp. 297–308.
136. Pereira, J.D.A.; Lourenço, N.; Vieira, M. On the Use of Deep Graph CNN to Detect Vulnerable C Functions. In Proceedings of the 11th Latin-American Symposium on Dependable Computing, Fortaleza, Brazil, 21–24 November 2022; pp. 45–50.
137. Dam, H.K.; Tran, T.; Pham, T.; Ng, S.W.; Grundy, J.; Ghose, A. Automatic feature learning for vulnerability prediction. *arXiv* **2017**, arXiv:1708.02368. [[CrossRef](#)]
138. Gear, J.; Xu, Y.; Foo, E.; Gauravaram, P.; Jadidi, Z.; Simpson, L. Software Vulnerability Detection Using Informed Code Graph Pruning. *IEEE Access* **2023**, *11*, 135626–135644. [[CrossRef](#)]
139. Li, Y.; Wang, S.; Nguyen, T.N. Vulnerability detection with fine-grained interpretations. In Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Athens, Greece, 23–28 August 2021; pp. 292–303.
140. Li, Y.; Yadavally, A.; Zhang, J.; Wang, S.; Nguyen, T.N. Commit-level, neural vulnerability detection and assessment. In Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, San Francisco, CA, USA, 3–9 December 2023; pp. 1024–1036.
141. Zhang, H.; Bi, Y.; Guo, H.; Sun, W.; Li, J. ISVSF: Intelligent vulnerability detection against Java via sentence-level pattern exploring. *IEEE Syst. J.* **2021**, *16*, 1032–1043. [[CrossRef](#)]
142. Peng, H.; Mou, L.; Li, G.; Liu, Y.; Zhang, L.; Jin, Z. Building program vector representations for deep learning. In Proceedings of the International Conference on Knowledge Science, Engineering and Management, Chongqing, China, 28–30 October 2015; Springer International Publishing: Cham, Switzerland, 2015; pp. 547–553.
143. Lee, Y.J.; Choi, S.H.; Kim, C.; Lim, S.H.; Park, K.W. Learning binary code with deep learning to detect software weakness. In Proceedings of the KSII the 9th International Conference on Internet (ICONI) 2017 Symposium, Vientiane, Laos, 17–20 December 2017.
144. Mikolov, T.; Chen, K.; Corrado, G.; Dean, J. Efficient estimation of word representations in vector space. *arXiv* **2013**, arXiv:1301.3781. [[CrossRef](#)]
145. TMikolov, I.; Sutskever, K.; Chen, C.; Corrado, G.S.; Dean, J. Distributed representations of words and phrases and their compositionality. In Proceedings of the Advances in Neural Information Processing Systems, Lake Tahoe, NV, USA, 5–10 December 2013; pp. 3111–3119.
146. Black, P.E.; Black, P.E. *Juliet 1.3 Test Suite: Changes From 1.2*; US Department of Commerce, National Institute of Standards and Technology: Gaithersburg, MD, USA, 2018.

147. Alenezi, M.; Zagane, M.; Javed, Y. Efficient deep features learning for vulnerability detection using character n-gram embedding. *Jordanian J. Comput. Inf. Technol. (JJCIT)* **2021**, *7*, 25–38. [\[CrossRef\]](#)
148. Sun, H.; Liu, Y.; Ding, Z.; Xiao, Y.; Hao, Z.; Zhu, H. An enhanced vulnerability detection in software using a heterogeneous encoding ensemble. In Proceedings of the 2023 IEEE Symposium on Computers and Communications (ISCC), Gammarth, Tunisia, 9–12 July 2023; IEEE: New York, NY, USA, 2023; pp. 1214–1220.
149. Tian, J.; Zhang, J.; Liu, F. BBregLocator: A vulnerability detection system based on bounding box regression. In Proceedings of the 2021 51st Annual IEEE/IFIP International Conference on Dependable Systems and Networks Workshops (DSN-W), Taipei, Taiwan, 21–24 June 2021; IEEE: New York, NY, USA, 2021; pp. 93–100.
150. Cheng, G.; Luo, Q.; Zhang, Y. Vulnerability detection with feature fusion and learnable edge-type embedding graph neural network. *Inf. Softw. Technol.* **2025**, *181*, 107686. [\[CrossRef\]](#)
151. Sun, Y.; Wu, D.; Xue, Y.; Liu, H.; Ma, W.; Zhang, L.; Liu, Y.; Li, Y. Llm4vuln: A unified evaluation framework for decoupling and enhancing llms' vulnerability reasoning. *arXiv* **2024**, arXiv:2401.16185. [\[CrossRef\]](#)
152. Lyu, Q.; Havaladar, S.; Stein, A.; Zhang, L.; Rao, D.; Wong, E.; Apidianaki, M.; Callison-Burch, C. Faithful chain-of-thought reasoning. In Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics (IJCNLP-AACL 2023), Nusa Dua, Bali, Indonesia, 1–4 November 2023.
153. Kojima, T.; Gu, S.S.; Reid, M.; Matsuo, Y.; Iwasawa, Y. Large language models are zero-shot reasoners. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 22199–22213.
154. Gao, Z.; Wang, H.; Zhou, Y.; Zhu, W.; Zhang, C. How far have we gone in vulnerability detection using large language models. *arXiv* **2023**, arXiv:2311.12420. [\[CrossRef\]](#)
155. Yao, Y.; Duan, J.; Xu, K.; Cai, Y.; Sun, Z.; Zhang, Y. A survey on large language model (llm) security and privacy: The good, the bad, and the ugly. *High-Confid. Comput.* **2024**, *4*, 100211. [\[CrossRef\]](#)
156. Zhou, X.; Cao, S.; Sun, X.; Lo, D. Large language model for vulnerability detection and repair: Literature review and the road ahead. *ACM Trans. Softw. Eng. Methodol.* **2025**, *34*, 1–31. [\[CrossRef\]](#)
157. Ding, Y.; Fu, Y.; Ibrahim, O.; Sitawarin, C.; Chen, X.; Alomair, B.; Wagner, D.; Ray, B.; Chen, Y. Vulnerability detection with code language models: How far are we? *arXiv* **2024**, arXiv:2403.18624. [\[CrossRef\]](#)
158. Guo, Y.; Patsakis, C.; Hu, Q.; Tang, Q.; Casino, F. Outside the comfort zone: Analysing llm capabilities in software vulnerability detection. In Proceedings of the European Symposium on Research in Computer Security, Bydgoszcz, Poland, 16–20 September 2024; Springer Nature: Cham, Switzerland, 2024; pp. 271–289.
159. Haurigné, J.; Basheer, N.; Islam, S. Advanced Vulnerability Detection Using Llm with Transparency Obligation Practice Towards Trustworthy Ai. *Mach. Learn. Appl.* **2024**, *18*, 100598.
160. Purba, M.D.; Ghosh, A.; Radford, B.J.; Chu, B. Software vulnerability detection using large language models. In Proceedings of the 2023 IEEE 34th International Symposium on Software Reliability Engineering Workshops (ISSREW), Florence, Italy, 9–12 October 2023; IEEE: New York, NY, USA, 2023; pp. 112–119.
161. Yang, A.Z.; Le Goues, C.; Martins, R.; Hellendoorn, V. Large language models for test-free fault localization. In Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, Lisbon, Portugal, 14–20 April 2024; pp. 1–12.
162. Du, X.; Wen, M.; Zhu, J.; Xie, Z.; Ji, B.; Liu, H.; Shi, X.; Jin, H. Generalization-enhanced code vulnerability detection via multi-task instruction fine-tuning. *arXiv* **2024**, arXiv:2406.03718.
163. Boi, B.; Esposito, C.; Lee, S. Smart contract vulnerability detection: The role of large language model (llm). *ACM SIGAPP Appl. Comput. Rev.* **2024**, *24*, 19–29. [\[CrossRef\]](#)
164. Yin, X.; Ni, C.; Wang, S. Multitask-based evaluation of open-source llm on software vulnerability. *IEEE Trans. Softw. Eng.* **2024**, *50*, 3071–3087. [\[CrossRef\]](#)
165. Ferrer, L. Analysis and comparison of classification metrics. *arXiv* **2022**, arXiv:2209.05355.
166. Muntean, M.; Militaru, F.D. Metrics for evaluating classification algorithms. In *Education, Research and Business Technologies: Proceedings of the 21st International Conference on Informatics in Economy (IE 2022)*, Bucharest, Romania, 26–27 May 2022; Springer Nature: Singapore, 2023; pp. 307–317.
167. Leiter, C.; Lertvittayakumjorn, P.; Fomicheva, M.; Zhao, W.; Gao, Y.; Eger, S. Towards explainable evaluation metrics for natural language generation. *arXiv* **2022**, arXiv:2203.11131. [\[CrossRef\]](#)

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.